

Amplitude-Only FFN Intervention for Tool-Structured LLM Inference Method: Gated Evaluation Protocol, and Cross-Model Empirical Results

Sheng Xu

xusheng.xs@alibaba-inc.com

Boyuan Huang

boyuan.hby@alibaba-inc.com

Ke Jia

jiake.jk@alibaba-inc.com

Jiadun Zhu

jifeng.zjd@taobao.com

Zhen Chen

moriarty.cz@alibaba-inc.com

Alibaba Cloud

Abstract

Large language models increasingly operate as tool-using agents, where small format, argument, or function-call errors can invalidate an otherwise plausible response. We study inference-time feed-forward network (FFN) intervention as a way to improve structured outputs without retraining model weights. The project began with Orthogonal Residual Projection (ORP), an early project-specific inference-time repair attempt that used SVD-derived orthogonal projection to modify residual-stream FFN representations. ORP was scientifically useful but not deployable. It exposed useful structure inside SwiGLU FFNs, including sensitive intervention positions P1, P2, and P3 and non-monotonic dependence on an energy parameter e . However, diagnostic runs showed that harm could exceed recovered fixes, suggesting that pretrained FFN directions encode semantic priors that should not be overwritten at inference time. We therefore propose Amplitude Gating (AG), a non-destructive alternative that preserves pretrained FFN weight directions and modulates only activation magnitudes during generation. We define a fine-grained intervention system for SwiGLU FFNs, covering inherited points P1/P2/P3 and structurally refined points P1s/P2a/P2b, and evaluate entropy-based AG candidates with deployable sample-level gates. A central methodological contribution is an evaluation protocol that separates combination-oracle headroom from fixed configurations and learned gates constrained to online-observable features, avoids pooled multi-configuration accounting, and uses task-aware metrics for strict binary versus partial-credit datasets. Across Qwen-family experiments spanning a hybrid linear/full-attention model (Qwen3.5-9B) and dense full-attention models (Qwen3-8B and Qwen2.5-7B), AG is weakly positive in aggregate but strongest on tool-structured tasks. On Qwen3.5-9B, a category-level learned gate improves tool/structured/agent tasks from 38.66% to 42.92% (+4.27 percentage points), with Hermes function-call tasks reaching about +7.6 pp learned-gate lift. On Qwen3-8B, the strongest signal shifts to JSON-structured output, where Hermes JSON mode improves from 41.83% to 53.19% (+11.36 pp). A Qwen2.5-7B follow-up shows that oracle headroom can exist even when current learned gates fail, indicating that AG should be deployed with model- and category-specific routing rather than a universal gate. Fixed-pool comparisons between non-Newton-Schulz entropy AG and Newton-Schulz-windowed AG further show that neither family is uniformly dominant: non-NS variants are sharper current-token amplitude gates, whereas NS variants add local-trajectory diversity whose value changes with model architecture and task category. These results identify tool-structured inference as the most credible first deployment target for safe FFN-level inference optimization. New feature-family ablations with bootstrap confidence intervals support the Qwen3/Qwen3.5 tool-route gains while confirming Qwen2.5-7B as a boundary case rather than a deployable positive route; prospective online validation, broader cross-model generalization, and independent overfitting checks remain required follow-up work.

1. Introduction

Tool use changes the error surface of large language models (LLMs) [7]. In ordinary open-ended generation, an answer may remain useful even when phrasing or some minor detail changes. In tool-calling and agentic settings, however, the output is often consumed by a parser, validator, API, or downstream program. A missing argument, invalid JSON bracket, wrong function name, or malformed schema can turn a semantically plausible answer into a failed action. This makes tool-structured inference a high-value target for inference-time optimization: small improvements can translate directly into more successful calls and fewer agent failures.

This paper studies a narrow but important question: can we improve LLM behavior by intervening inside the feed-forward network (FFN) at inference time while preserving the pretrained model's semantic structure? FFN layers are attractive because they are present across a wide range of Transformer-like models and are often understood as the layers where token-wise nonlinear feature transformations are expressed. A method that operates at the FFN level can therefore be more architecture-general than a technique tied to a specific attention layout. At the same time, FFN intervention is risky: any method that changes internal representations can repair some samples while damaging others. For deployment, the relevant question is not whether an intervention can sometimes help, but whether it can be accepted safely when it is likely to help and rejected otherwise.

The core intuition is that attention brings information into context, while the FFN transforms that information into the feature pattern from which the next output preference is formed. Tool-structured failures often look like last-mile

transformation errors: the model is near the right semantic region but emits the wrong argument boundary, function endpoint, or schema continuation. This makes the FFN a natural place to seek a conservative intervention surface.

This architecture-general motivation must be stated carefully. The experiments in this paper do not prove that one universal AG gate transfers unchanged across all LLMs. They support a weaker and more defensible claim: because FFN blocks appear across Transformer-like LLMs [1], a non-destructive FFN amplitude intervention can be mapped to a broad class of models even when their attention mechanisms differ. In this paper, the concrete intervention taxonomy is instantiated on SwiGLU-style gated FFNs [2], because the evaluated Qwen checkpoints use gate/value branches. This is an implementation-specific taxonomy rather than a limit of the core AG principle: P1/P3-style input/output amplitude gating and P2-like internal activation gating can be adapted to other FFN variants, while P1s/P2a/P2b specifically exploit SwiGLU's branch structure. Our current evidence spans Qwen3.5-9B, an evaluated Qwen-family checkpoint whose text stack mixes linear attention and full attention in a repeating 3-to-1 pattern, and dense full-attention Qwen-family models such as Qwen3-8B [3] and Qwen2.5-7B [4]. Positive learned-gate results on Qwen3.5-9B and Qwen3-8B support cross-structure existence of the AG effect, while the weaker Qwen2.5-7B gate result shows that deployment still requires model- and category-specific calibration.

Our project began with Orthogonal Residual Projection (ORP), implemented in the project archive as a dynamic per-token SVD-based projection intervention. ORP was an internal exploratory method rather than an established external baseline. In ORP, "residual" refers to the intermediate representation carried by the model residual stream, and "orthogonal projection" refers to using singular value decomposition (SVD) to construct projection directions or matrices that modify that representation at selected model sites. ORP was useful scientifically but not deployable. It identified sensitive FFN positions, later called P1, P2, and P3, and showed that an energy-like parameter e has non-monotonic effects on fix and harm. In one archived P1 sweep over 1,319 GSM8K examples, for example, $e=0.93$ produced 21 benefits and 12 harms, whereas more aggressive or more conservative values could reduce or reverse the net gain. The same archive also shows that the project explicitly supported FFN input, FFN internal, and FFN output projection modes. Early compression-oriented ORP tests were even more severe: a baseline MMLU score of about 79.1% fell to roughly random-choice levels under ORP_Attn (24.1%), ORP_FFN_Static (24.3%), and ORP_FFN_Dyn (25.0%). Because those earliest tests may contain pipeline-specific bugs, we use them only as a coarse diagnostic that direction-changing projection can destroy model utility. In a later full-GSM8K matrix audit, the loaded SVD-ORP checkpoint `qwen35|svd_orp|p1|e0.95|w3` covered 1,032 common questions and produced 34 fixes but 56 regressions relative to the baseline, with 1,012/1,032 generations diverging at the token level. We treat this partially completed checkpoint as a failure diagnostic rather than a main benchmark, but it captures the central lesson: pretrained FFN directions appear to carry semantic priors and should not be overwritten by a static direction-changing repair.

We therefore move from direction modification to amplitude modulation. We call the resulting method Amplitude Gating (AG). AG preserves the pretrained FFN weight directions and instead clips or gates activation magnitudes during inference. This is a deliberately conservative design. It keeps the useful insight from ORP, namely where the FFN is sensitive and how energy thresholds affect behavior, while avoiding the destructive part of ORP, namely changing matrix or subspace directions. In deployment, AG is paired with a gate: an AG candidate is accepted only when observable features predict that the candidate is more likely to fix than harm; otherwise generation falls back to the baseline model.

We make five contributions.

First, we formulate AG as a non-destructive FFN intervention and define a fine-grained intervention system for SwiGLU FFNs. The system includes inherited positions P1, P2, and P3 from the ORP stage and refined positions P1s, P2a, and P2b that separate gate-branch, value-branch, fused, and output-side signals.

Second, we identify evaluation pitfalls that can substantially overstate inference-time intervention quality. In particular, pooled multi-configuration accounting, best-selected oracle accounting, and row-level rather than sample-level accounting can make a method look deployable when it is only an offline upper bound. We distinguish combination-oracle headroom from best fixed single-configuration and learned-gate offline results.

Third, we show that AG is not uniformly strong across all text tasks. Its most credible gains concentrate in tool-structured settings. On Qwen3.5-9B, tool/structured/agent category performance improves from 38.66% to 42.92% under a learned category gate (+4.27 pp), with Hermes function-call datasets showing learned-gate gains of about +7.6 pp. On Qwen3-8B, the strongest result shifts to JSON-structured output, where Hermes JSON mode improves by +11.36 pp.

Fourth, we show that cross-model existence does not imply a universal gate. Qwen2.5-7B has measurable oracle headroom, including +4.03 pp on tool-structured tasks, but current learned gates do not capture it at the category level. The feature-family ablation adds bootstrap confidence intervals to this boundary: Qwen3/Qwen3.5 tool-route feature gates are CI-positive, whereas Qwen2.5-7B is not. This makes Qwen2.5-7B an important boundary case: AG candidates can contain useful alternatives, but model-specific routing and stronger features are required before deployment.

Finally, we distinguish non-NS AG from Newton-Schulz-windowed NS AG. The fixed comparisons do not support a single winner. Qwen3.5-9B favors non-NS on tool/structured rows but NS on Long QA and factuality; Qwen3-8B and

Qwen2.5-7B show different patterns. We interpret this as a model-by-task interaction over FFN activation trajectories: non-NS is the sharper current-token amplitude family, while NS adds short-window local-trajectory scoring.

2. Related Work

AG sits between several active lines of work, but its goal and operating level are different from each of them.

FFN sparsification and acceleration. Fast feed-forward and conditional FFN methods reduce inference cost by executing fewer neurons or replacing dense FFN computation with cheaper conditional computation [12]. AG is not primarily a speed method. It deliberately keeps the pretrained FFN weights and uses amplitude masking as a candidate-generating intervention whose value is judged by fix/harm and deployable gate behavior. This makes AG complementary to acceleration work: a faster FFN does not by itself decide when a modified activation is safer than the baseline.

Inference-time activation steering. Representation engineering and steering methods manipulate model activations to induce high-level behavioral changes [13]. AG is narrower. It does not inject a task-specific semantic direction or optimize for a behavioral attribute such as honesty or sentiment. Instead, it preserves pretrained FFN weight directions and changes only the amplitude distribution at identified FFN sites, then relies on an online gate to reject samples where the candidate appears risky.

Structured output decoding. Constrained decoding and JSON-schema-guided generation enforce output syntax at the decoder level [14]. These methods are important for production systems, but they operate after the model has already computed the next-token distribution. AG instead intervenes inside FFN computation before decoding. It therefore targets a different failure mode: cases where internal activation changes can alter which tool, function, or argument structure the model prefers, not only whether the final string obeys a grammar.

Tool-use and tool-learning systems. ToolBench and ToolLLM frame tool use as data construction, training, retrieval, and evaluation over API calls [6]. AG does not replace tool training, retrieval, or tool-specific prompting. It is an inference-time candidate mechanism for models already asked to produce tool-structured outputs. Its current evidence is strongest exactly in this setting, but the deployment claim remains conditional on model- and category-specific gates rather than a universal tool-use improvement.

3. Method: From the FFN Intervention Hypothesis to Amplitude Gating

3.1 The FFN intervention hypothesis

We begin from a simple internal question: when a tool-structured output fails, where does the failure become a preference for the wrong token, argument, function name, or schema? Decoder-level constraints can enforce syntax, and prompting can bias the final distribution, but neither explains how the model's internal computation turns a plausible instruction into a malformed or incorrect structured action. We therefore focus on the feed-forward network (FFN), the token-wise nonlinear transformation module that appears across Transformer-like LLMs.

This choice is motivated by both mechanism and generality. Attention brings information into context, while the FFN transforms that information into the features from which the next output preference is formed. In tool calling, JSON generation, and other structured outputs, many failures are not simply cases where the model lacks relevant information. They are cases where the model appears to have the right semantic neighborhood but produces a wrong boundary, wrong argument, wrong endpoint, or malformed continuation. These are natural candidates for a local transformation error inside the FFN stack.

The FFN is also an attractive target because it is less tied to a particular attention architecture. Attention mechanisms vary across dense full attention, hybrid attention, linear attention, and emerging sequence models, but FFN-like nonlinear channel-mixing blocks remain a common substrate in modern LLMs. A safe FFN-level intervention could therefore open an optimization surface that does not require retraining model weights and is not specific to one decoding algorithm or attention layout.

The difficulty is safety. An internal intervention that sometimes repairs a structured output can also damage samples that the baseline model already solves. The method must therefore answer two questions at once: where inside the FFN should we intervene, and what form of intervention respects the pretrained model's semantic structure? Our first attempt answered the first question but failed the second. That attempt was Orthogonal Residual Projection.

3.2 ORP as the first FFN repair attempt

The original ORP direction tested a direct directional hypothesis: if an FFN representation drifts toward an undesirable output, an SVD-derived orthogonal projection might move it back into a better subspace. The implementation archive describes the method as "Dynamic per-token Orthogonal Residual Projection." In this paper we use the shorter name ORP for that internal project method. Its intervention modes include FFN input, FFN internal, FFN output, attention, and all-linear projections. In the FFN case, the relevant positions were:

- P1: after normalization and before the up/gate projections, corresponding to FFN input.
- P2: around the internal FFN expansion/fusion region.
- P3: after the down projection and before residual addition, corresponding to FFN output.

The ORP stage provided diagnostic evidence that motivates the rest of this paper. It established three facts.

First, the FFN is not uniformly sensitive. P1, P2, and P3 differ in noise, signal strength, dimensionality, and deployment risk. This gave us a natural coordinate system for later AG experiments.

Second, e-value controls are non-monotonic. In archived P1 experiments over 1,319 examples, $e=0.93$ was the best P1 setting among the inspected values (accuracy 92.87%, benefit/harm 21/12, net +9), while $e=0.97$ was net negative (benefit/harm 11/16, net -5) and $e=0.80$ was also net negative (29/31, net -2). This supports treating e not as a monotonic "more compression is better" parameter, but as a tunable trade-off between preserving useful signal and suppressing harmful amplitude.

Third, the fix/harm distribution is sample-specific. The same P1 sweep found 1,137 stable-correct samples, 54 stable-wrong samples, and 128 flipping samples; among flipping samples, 49 were benefit-only and 79 were harm-only. That imbalance is exactly the reason that an inference-time method needs a gate.

3.3 Negative result as design principle

ORP was not deployable, but it was diagnostic. It showed that FFN intervention is position-dependent, energy-dependent, and sample-dependent. More importantly, it suggested what must not be changed. The goal was to repair FFN representations by correcting subspace directions, but the pretrained model's directions are not arbitrary. They appear to be part of the learned semantic geometry on which downstream layers expect to operate. When those directions are changed, the model can move away from that geometry. In practice, this led to the central project conclusion: matrix-direction changes can produce more harm than fix.

A small completed ORP sweep adds a fully observed failure diagnostic before the larger matrix audit. In this sweep, the baseline score was 85% over 20 samples. At the P1 residual-stream site before `up_proj`, high-energy ORP fell to 30% at $e=0.95$ and 50% at $e=0.85$, while $e=0.70$ recovered the baseline; the low-energy direction was even more brittle, falling to 10% at $e=0.95$ and 50% at $e=0.85$ before also returning to 85% at $e=0.70$. At the P2 activation-side site before `down_proj`, the intervention collapsed to 5% for the high-energy $e=0.95$ setting and to 10%, 10%, and 50% for the low-energy $e=0.95$, $e=0.85$, and $e=0.70$ settings. Because $n=20$ is small, we treat this sweep only as qualitative evidence. Its value is diagnostic: ORP can severely damage model behavior, P2/down-projection interventions are especially risky, and milder e settings sometimes recover baseline, reinforcing non-monotonic e sensitivity rather than making ORP deployable.

The clearest direction-changing diagnostic comes from the full-GSM8K matrix audit. The loaded SVD-ORP run `qwen35|svd_orp|p1|e0.95|w3` was incomplete, so we do not use it as a primary performance result. Still, on its 1,032 common questions it shows the failure mode sharply: SVD-ORP improved 34 baseline-wrong answers but damaged 56 baseline-correct answers, for a net -22 strict balance. Its strict accuracy was 85.27%, below the 87.72% Qwen3.5 baseline reported in the same matrix, and 1,012 of 1,032 generations diverged from the baseline token path. This is exactly the behavior that makes direction correction unsafe for deployment: it can find real repairs, but the repair operation is too invasive.

ORP exposes FFN sensitivity, but direction-changing repair is unsafe

Remote ORP archive: GSM8K diagnostics from /mnt/data/dyn_orp

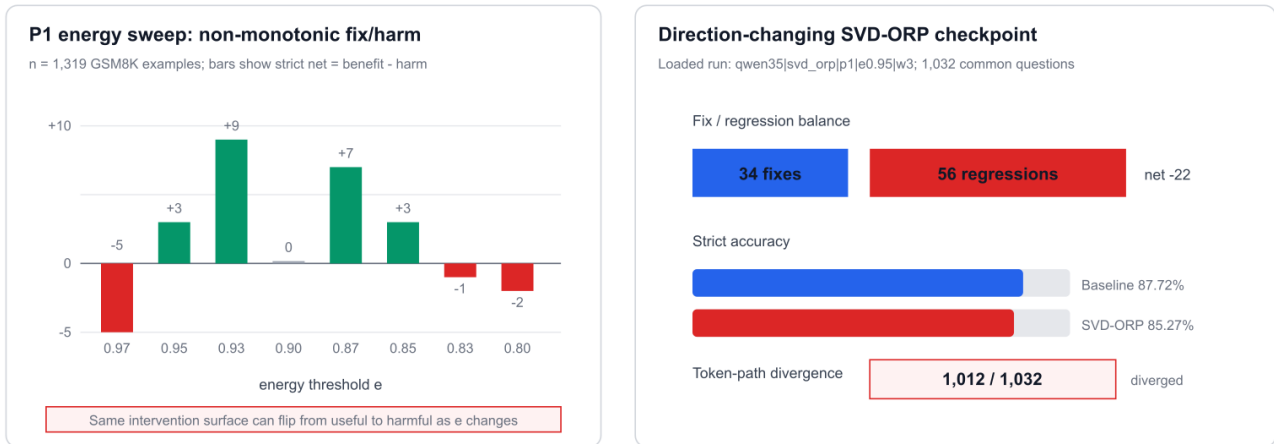


Figure 1. ORP failure diagnostic: P1 energy sweeps are non-monotonic, and the loaded SVD-ORP checkpoint produces more regressions than fixes.

This negative result is the conceptual pivot of the paper. It does not imply that FFN intervention is impossible. It implies that an inference-time FFN intervention should not rotate or overwrite pretrained FFN weight directions. Those directions should be treated as semantic structure rather than adjustable noise. Instead of rewriting the projection basis used by the model, we should keep the pretrained weights fixed and only adjust activation magnitudes when there is evidence that the current activation is over- or under-amplified.

3.4 AG as the conservative successor

Amplitude Gating adopts this conservative principle. Given an FFN activation x at a selected intervention point, AG computes a gated activation x' by masking or shrinking selected activation dimensions while leaving all FFN weight matrices unchanged. This does not guarantee that the activation vector direction itself is unchanged; hard and top-k masks can rotate the activation vector. The conservative claim is narrower: AG preserves pretrained weight directions and changes only activation amplitudes. In the current experiments, the main family is entropy-based AG. Entropy provides a compact signal of distributional concentration: when an activation or token trajectory becomes too diffuse or too concentrated, entropy-derived gates can suppress unstable components or retain high-confidence components.

The important distinction is that AG is an inference-time activation operation, not a weight update. It does not retrain the model, change FFN matrices, or inject a task-specific steering vector. It computes an alternative forward path and then relies on a gate to decide whether that alternative should be used.

AG keeps the FFN coordinate system that ORP disrupts

The design pivot is from direction-changing repair to amplitude-only modulation plus fallback

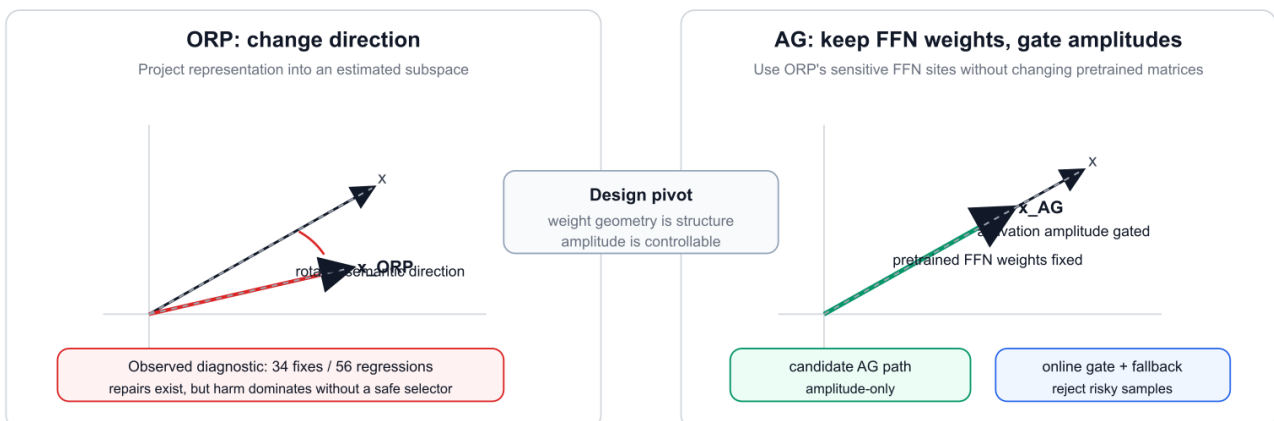


Figure 2. AG versus ORP design pivot: ORP changes SVD-derived projection directions, while AG keeps pretrained FFN weights fixed and modulates activation amplitudes under a gate-and-fallback policy.

3.5 FFN computation and the SwiGLU instantiation

AG intervention points inside an LLM FFN block

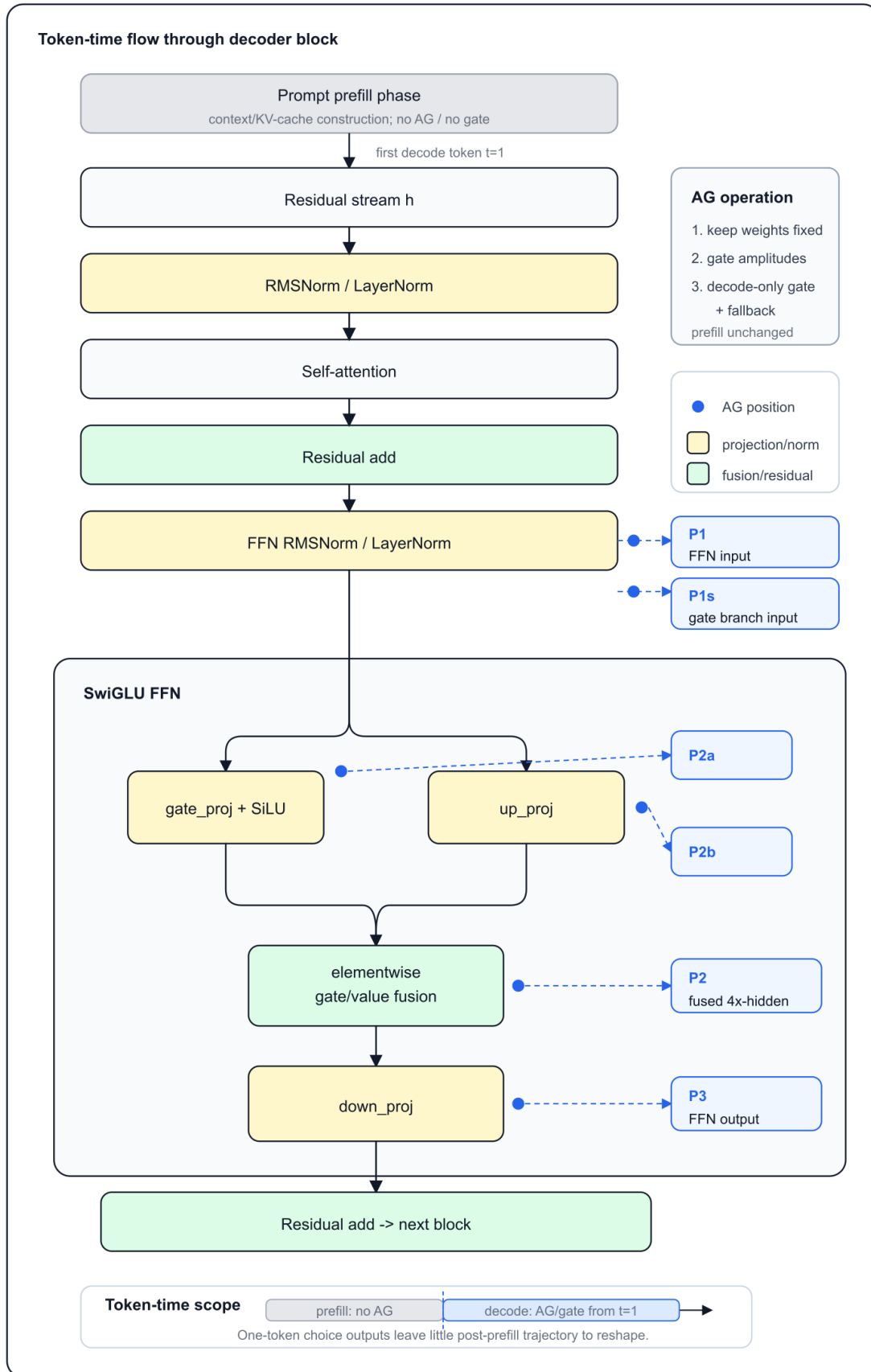


Figure 3. AG intervention points inside a Transformer FFN block. AG keeps FFN weights fixed and gates activation amplitudes at selected P-sites. The evaluated implementation is decode-only: prompt prefill is unchanged, and AG/gate decisions start at the first generated token.

AG is defined at the level of FFN activations rather than at the level of a particular attention mechanism. For the Qwen checkpoints studied here, the relevant FFN is a SwiGLU-style gated FFN. A simplified layer computation is:

```
h -> LayerNorm(h)
  -> gate_proj -> SiLU
  -> up_proj
  -> elementwise gate/value fusion
  -> down_proj
  -> residual addition
```

This structure exposes multiple intervention locations. Some are hidden-dimensional and cheap to observe or modify; others are expansion dimensional and more expensive but potentially more informative.

The distinction matters for generality. The AG principle applies to FFN activation amplitudes in general: an FFN input, an internal nonlinear activation, and an FFN output can be gated without changing weight directions. The six-point taxonomy in this paper is the SwiGLU instantiation of that principle. P1 and P3 correspond to broadly available FFN input/output sites; P2 corresponds to an internal expanded activation; P1s, P2a, and P2b are refined sites made possible by SwiGLU's gate/value split.

3.6 Intervention points

We use the following intervention taxonomy. The priority columns separate two different decisions: which points are most informative for mechanism discovery, and which points are most attractive for low-risk deployment.

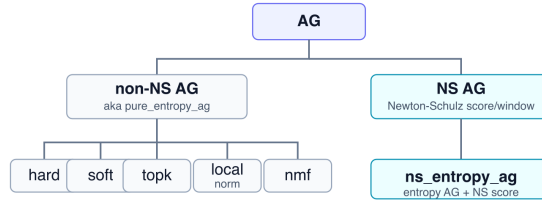
Point	Location	Dimensionality	Status	Research priority	Deployment priority	Rationale
P1	After layer norm, before up/gate projection	hidden	Verified inherited point	Medium	Medium	Low-cost standardized FFN input; useful baseline point but mixes gate and value branches
P1s	Gate-branch-only input variant	hidden	Tested in new-P candidate space; broader validation ongoing	High	High	Targets gate-branch selection with hidden-size cost; promising bridge between mechanism and deployment
P2a	After gate projection and SiLU, before fusion	4x hidden	Tested in new-P candidate space; broader validation ongoing	High	Low	Direct channel-switch signal, but expansion-dimensional and therefore costly/noisier online
P2b	After up projection, before fusion	4x hidden	Tested in new-P candidate space; broader validation ongoing	High	Low-Medium	Strong value-branch evidence on some tool tasks, but still expansion-dimensional
P2	After gate/value fusion, before down projection	4x hidden	Verified inherited point	Highest	Low	Most information-rich fused FFN representation, but high cost and high noise make it mainly a research/oracle point
P3	After down projection, before residual addition	hidden	Verified inherited point	Medium-High	Highest	Clean output-side hidden-size signal before residual addition; best first choice for conservative deployment

P1/P2/P3 are inherited from the ORP stage. P1s/P2a/P2b are structural refinements introduced after the transition to AG. They are included because SwiGLU separates gate-branch, value-branch, fused, and output-side computation; the result section reports which of these points become useful under concrete model/task routes. Claims about P1s/P2a/P2b as universally optimal or mechanistically orthogonal remain future work.

This ranking leads to a two-track experimental strategy. For mechanism and oracle-space analysis, P2/P2a/P2b should remain high priority because they observe the expanded FFN computation most directly. For deployable AG, the preferred order is P3 first, then P1s, then task-specific P1/P2b variants only when held-out evidence justifies the extra risk or cost. We therefore avoid claiming that the most informative point is automatically the best production point.

3.7 AG taxonomy, formulas, and comparison

AG is a family of non-destructive activation interventions. All variants operate on an FFN activation vector at inference time and leave FFN weight directions unchanged. We use the following taxonomy:



Let x_t in $\mathbb{R}^d$ be the activation at token t and a selected intervention point. All mask-style variants return $x'_t = x_t$ masked by a selected dimension set. For the energy and entropy variants, the parameter e denotes retained mass: retained L2 energy for hard/soft/NMF-style gates and retained entropy mass for NS AG. Local-norm AG instead uses e as a standard-deviation multiplier, while top-k AG uses an explicit dimension budget k . A cleaner implementation would rename these as ρ for retained mass, λ for the local-norm multiplier, and k for top-k; we keep the archive notation e in the reported run keys. For a selected set K , $M_K(i)$ denotes the binary mask that equals 1 when i is in K and 0 otherwise. In this taxonomy, `pure_entropy_ag` is the umbrella name for the non-NS entropy-ranking branch rather than an additional leaf variant.

Formula block (1): Hard AG. Hard AG is the direct L2-energy gate. It sorts dimensions by squared magnitude and keeps the shortest descending-magnitude prefix whose cumulative energy reaches the target retained mass:

$$\begin{aligned}
 E &= \sum_i x_{t,i}^2 \\
 \pi &= \text{argsort}(x_{t,i}^2, \text{descending}) \\
 k_{\text{hard}} &= \min \left\{ k : \sum_{r=1}^k x_{t,\pi_r}^2 \geq eE \right\} \\
 K_{\text{hard}} &= \{\pi_1, \dots, \pi_{k_{\text{hard}}}\} \\
 x'_{t,i} &= x_{t,i} M_{K_{\text{hard}}}(i)
 \end{aligned} \tag{1}$$

Formula block (2): Soft AG. Soft AG uses the same L2-energy ranking to choose an adaptive threshold τ , but shrinks rather than simply masking:

$$\begin{aligned}
 \tau &= |x_{t,\pi_{k_{\text{hard}}}}| \\
 x'_{t,i} &= \text{sign}(x_{t,i}) \max(|x_{t,i}| - \tau, 0)
 \end{aligned} \tag{2}$$

Formula block (3): Top-k AG. Top-k AG fixes the number of retained dimensions instead of deriving it from an energy threshold:

$$\begin{aligned}
 K_{\text{topk}} &= \text{TopK}_i(|x_{t,i}|, k) \\
 x'_{t,i} &= x_{t,i} M_{K_{\text{topk}}}(i)
 \end{aligned} \tag{3}$$

Formula block (4): Local-norm AG. Local-norm AG uses a local standard-deviation threshold. It is adaptive to the scale of the current activation vector, but does not sort dimensions:

$$\begin{aligned}
 \sigma_t &= \text{std}(x_t) \\
 K_{\text{local}} &= \{i : |x_{t,i}| > e\sigma_t\} \\
 x'_{t,i} &= x_{t,i} M_{K_{\text{local}}}(i)
 \end{aligned} \tag{4}$$

Formula block (5): NMF-style AG. NMF-style AG splits positive and negative activation components, applies the same L2-energy prefix rule to each non-negative half, and recombines them. This preserves sign structure while treating additive positive and negative evidence separately:

$$\begin{aligned}
x_t^+ &= \max(x_t, 0), & x_t^- &= \max(-x_t, 0) \\
E_v &= \sum_i v_i^2, \\
\pi^v &= \text{argsort}(v_i^2, \text{descending}) \\
k_v &= \min\{k : \sum_{r=1}^k v_{\pi_r^v}^2 \geq eE_v\} \\
K_v &= \{\pi_1^v, \dots, \pi_{k_v}^v\} \\
[g(v, e)]_i &= v_i M_{K_v}(i) \\
x'_t &= g(x_t^+, e) - g(x_t^-, e)
\end{aligned} \tag{5}$$

Formula block (6): NS AG. NS AG is the second branch. It keeps the AG mask interface but replaces the direct non-NS selection signal with an entropy-prefix mask whose dimension order is computed by a Newton-Schulz score from a recent token window. Define the entropy mass

$$\begin{aligned}
a_i &= |x_{t,i}| \\
p_i &= \frac{a_i}{\sum_j a_j + \varepsilon} \\
m_i &= -p_i \log(p_i + \varepsilon) \\
H &= \sum_i m_i
\end{aligned} \tag{6}$$

and the Newton-Schulz window score

$$\begin{aligned}
W_t &= [x_{t-w+1}; \dots; x_t] \in \mathbb{R}^{w \times d} \\
Y_0 &= \frac{W_t}{\|W_t\|_F + \varepsilon} \\
Y_{r+1} &= \frac{1}{2}(3Y_r - Y_r Y_r^\top Y_r), \quad r = 0, \dots, R-1 \\
s_t^{\text{NS}} &= \text{abs}(\text{lastrow}(Y_R)) \\
\pi &= \text{argsort}(s_t^{\text{NS}}, \text{descending}) \\
k_{\text{NS}} &= \\
\min\{k : \sum_{r=1}^k m_{\pi_r} \geq eH\} \\
K_{\text{NS}} &= \{\pi_1, \dots, \pi_{k_{\text{NS}}}\} \\
x'_{t,i} &= x_{t,i} M_{K_{\text{NS}}}(i)
\end{aligned} \tag{7}$$

The window is important because a single-vector Newton-Schulz score largely collapses back toward magnitude ranking in this implementation, whereas a short window lets the score depend on the local token trajectory. NS AG is therefore a ranking-score variant of entropy AG, not an ORP/SVD projection. The absolute-value operation above is elementwise, so $s_t^{\text{NS}} \in \mathbb{R}^d$ and can be used for dimension-wise sorting. NS AG does not construct a projection basis and does

not apply $x @ V_r @ V_r.T$. The Newton-Schulz component is motivated by Newton-Schulz-style matrix normalization and by project-archive notes on recent optimizer uses of Newton-Schulz iteration, including Muon/HybridNS-style usage discussed around DeepSeek-V4 [5]. Our use is different: those optimizer settings apply Newton-Schulz during training, whereas NS AG uses a short-window Newton-Schulz score only at inference time to rank FFN activation dimensions before applying the same entropy-mass mask interface. Newton-Schulz is therefore not used here to compute entropy, update weights, or orthogonalize model matrices; it changes only the dimension ordering used by the entropy mask. Reference [5] is retained as project-archive context, not as public or peer-reviewed evidence for the AG claims.

Variant	Selection signal	Operation	State/window	Main use
hard	L2 energy prefix	Binary mask	Stateless	Simple energy-preserving sparsification
soft	L2-derived tau	Shrinkage	Stateless	Smoother but more destructive amplitude change
topk	Fixed top-k magnitude	Binary mask	Stateless	Controlled compute/keep budget
local_norm	Local std threshold	Binary mask	Stateless	Scale-adaptive heuristic
nmf	Positive/negative L2 prefixes	Signed recombination	Stateless	Separates positive and negative evidence
ns_entropy_ag	Entropy mass, NS-window rank	Binary mask	Recent token window	Main NS text-model candidate

3.8 Gate and fallback

AG is evaluated as a gated candidate, not as an unconditional replacement. For a prompt and model, we evaluate a baseline output and one or more AG candidate policies during offline mining. A deployable version would learn a policy using features that are available online. If the gate accepts, the AG output is used; otherwise the system falls back to baseline.

In the evaluated implementation, AG is decode-only. The prompt prefill is left unchanged, and the first possible intervention occurs at the first generated decode token. This design avoids perturbing the encoded prompt state, but it also defines an important boundary: tasks that reduce the answer to a one-token multiple-choice label have very little post-prefill trajectory for AG to reshape. We therefore treat multiple-choice results mainly as diagnostics for candidate-space headroom and safety, not as the primary setting where AG should show large effects.

This distinction is central. Offline combination oracles can choose the best candidate after seeing labels; deployed gates cannot. We therefore report oracle results only as headroom and learned/fixed results as deployable-feature-constrained offline evidence.

The baseline is always available as a fallback when a deployable gate rejects an AG candidate. It is excluded only from oracle candidate choice, so that oracle scores measure useful non-baseline AG alternatives rather than the trivial ability to keep the baseline whenever AG is harmful.

4. Experiment Design: Coverage and Credibility Protocol

4.1 Design goals: coverage without overclaiming

The experiment design has to satisfy two competing requirements. It must be broad enough to avoid optimizing for one accidental model, dataset, position, or e-value, but constrained enough that the resulting evidence can still be interpreted. We therefore separate two questions throughout the protocol:

1. Does the AG candidate space contain useful alternatives? 2. Can a fixed configuration or online-feature gate choose those alternatives without seeing the answer score?

The first question is answered by oracle and candidate-space diagnostics. The second question is answered only by best fixed single-configuration results or learned gates constrained to deployable features. This distinction is the main protection against overclaiming.

Axis	Coverage choice	Why it matters
Model architecture	Qwen3.5-9B hybrid linear/full attention; Qwen3-8B and Qwen2.5-7B dense full attention	Tests whether FFN-level AG remains relevant when attention structure changes
Task category	Tool-structured, long QA, hallucination/factuality proxy, plus MC diagnostics in earlier reports	Prevents a tool-only result from being mistaken for a universal text result
FFN position	P1/P2/P3 inherited from ORP; P1s/P2a/P2b added from SwiGLU structure; here P means position/site, not a statistical p-value	Separates input, gate-branch, value-branch, fused, and output-side effects

Axis	Coverage choice	Why it matters
AG configuration	Non-NS entropy variants, Newton-Schulz entropy variants, e-values such as 0.93/0.95, and windowed settings; for entropy AG, e is the retained entropy-mass target defined in Section 3.7	Measures sensitivity to amplitude threshold and local entropy ranking definition
Gate feature family	Prefix/format, early logprob trajectory, baseline uncertainty, divergence/energy, and diagnostic-only post-fork features	Distinguishes deployable gate features from explanatory but non-deployable signals

This coverage design is why the paper reports both positive and negative evidence. Qwen3.5-9B and Qwen3-8B provide positive deployable-feature- constrained learned-gate offline results in different structured subdomains, while Qwen2.5-7B exposes a failure mode where candidate headroom exists but current gates do not reliably capture it. That negative boundary is part of the design rather than an embarrassment: it prevents the method from being over-claimed as a universal switch.

4.2 Candidate matrix and intervention coverage

For each model and dataset, we evaluate an explicit baseline and a matrix of AG candidates. In the aligned text/tool mining runs, each sample is paired with the baseline output and 24 adjustable AG candidates: 12 old-P candidates and 12 new-P candidates. The old-P set covers the inherited ORP positions P1/P2/P3; the new-P set covers the refined SwiGLU positions P1s/P2a/P2b. Candidate families include non-NS entropy AG and Newton-Schulz entropy AG variants, with e values such as 0.93 and 0.95 and windowed settings where applicable.

The notation is deliberately literal. `P` denotes the FFN intervention position, not statistical significance. In run keys such as `p1s` or `p2b`, it tells us where the activation is read and masked. For the entropy candidates emphasized in the text/tool results, `e` denotes the retained entropy mass used to choose the mask prefix. Thus `e0.93` and `e0.95` are two retained-mass settings at the same FFN point. Each aligned old-P and new-P block is therefore a matched candidate design: three positions crossed with two e-values and two AG families (`pure_entropy_ag` and `ns_entropy_ag`, with `w4` for the NS window). This matched design is what allows the P-by-e comparison reported in Section 5 and analyzed in Section 6.1.

The e-values in the matched text/tool tables are not learned from the reported test labels. They are fixed before the route-level result comparison as a small retained-mass grid inherited from the ORP/early-AG sweep: earlier diagnostics showed that nearby retained-mass settings could change fix/harm balance, with values around 0.93–0.95 repeatedly appearing in the sensitive region. We therefore keep `e0.93` and `e0.95` as paired probes rather than selecting a new best e per model or per dataset. When a table says that a cell such as `P2b/e0.93` is strongest, that is a result within this pre-specified grid, not a claim that `e=0.93` is globally optimal.

We use two discrete e-values rather than a dense sweep for three practical reasons. First, the purpose of the matched matrix is interaction testing, not hyperparameter optimization: the question is whether position changes the effect at nearby retained-mass settings. Second, every additional e-value multiplies the candidate count by model, dataset, position, and AG family, increasing compute cost and the risk of post-hoc best-cell overinterpretation. Third, 0.93 and 0.95 form a compact bracket inside the previously observed sensitive region: close enough to test local stability, but separated enough to expose sign or fix/harm changes. A denser e sweep remains useful future work, especially for deployment calibration, but it is not used as the main evidence level in this paper.

Why e0.93 and e0.95 are paired probes, not a dense sweep



Figure 5. Matched e-grid design: `e0.93` and `e0.95` are paired probes inside the sensitive retained-mass region, so the experiment tests P-by-e interaction without turning the candidate matrix into a dense post-hoc e search.

The candidate matrix is intentionally balanced across AG morphology. For each position/e setting, the non-NS form ranks dimensions by the current activation magnitude, while the NS form ranks dimensions by a Newton-Schulz score. The windowed NS setting, written as w_4 in the run keys, uses the most recent four decode-token activations to estimate a local score for the current token. This design lets the experiment answer two separate questions: whether simple magnitude-ranked entropy AG is already sufficient, and whether local Newton-Schulz scoring adds useful candidate diversity. We do not treat NS as a strictly stronger replacement for non-NS AG; it is a parallel morphology whose value must be established by fixed-configuration, oracle-selection, and learned-gate evidence.

Baseline output is always retained as the evaluation reference. It is not eligible for best fixed single-configuration, combination oracle, or learned-gate candidate choices. This convention makes oracle delta interpretable: oracle results measure whether non-baseline AG candidates contain useful alternatives, not whether an oracle can simply choose the baseline when AG is harmful.

The feature matrix records both candidate outputs and online-observable features. The Qwen3.5-9B and Qwen3-8B aligned mining reports each use old-P and new-P AG candidate matrices, with 1,026 gate-policy features and 31 diagnostic-only features. Qwen3.5-9B covers 241,056 feature rows; Qwen3-8B covers 165,432 feature rows. The Qwen2.5-7B BS64 old-P and new-P run is complete and also covers 241,056 rows.

4.3 Models, datasets, and evaluation routes

The current evidence base covers three concrete checkpoints over eight text/tool datasets. We report checkpoint names rather than only family names because the architectural differences matter for interpreting generalization.

Experiment key	Checkpoint	Attention structure	FFN shape	Context config	Role in this paper
qwen35	Qwen3.5-9B	Hybrid text stack: 32 layers with 24 linear-attention layers and 8 full-attention layers; full attention every fourth layer	SwiGLU-style gated FFN; hidden 4096, intermediate 12288, SiLU	max position 262144	Strongest function-call/tool-use evidence
qwen3	Qwen3-8B	Dense full-attention causal LM; 36 layers	SwiGLU-style gated FFN; hidden 4096, intermediate 12288, SiLU	max position 40960	Strongest JSON/prefix-format evidence
qwen25_7b	Qwen2.5-7B	Dense full-attention causal LM; 28 layers	SwiGLU-style gated FFN; hidden 3584, intermediate 18944, SiLU	max position 131072	Boundary case: oracle headroom but weak current gate

This model set gives a useful but bounded architecture comparison. Qwen3.5-9B tests AG in a hybrid linear/full-attention architecture, while Qwen3-8B and Qwen2.5-7B test AG in dense full-attention architectures with different depth and FFN expansion sizes. All three use SwiGLU-style gated FFNs, which allows the same six-point taxonomy to be evaluated across models. The broader AG principle is not restricted to SwiGLU, however: it requires only identifiable FFN activation sites whose amplitudes can be modulated without changing weight directions. The empirical claim is therefore conservative. AG produces positive deployable-feature-constrained learned-gate offline gains on Qwen3.5-9B and Qwen3-8B, and exposes oracle headroom but not yet a reliable category-level gate on Qwen2.5-7B. This supports FFN-level applicability across attention architectures and FFN instantiations, while rejecting the stronger claim that one gate is universally transferable.

The datasets are grouped into three evaluation routes:

- `tool_structured_agentic`: ToolBench in-domain, ToolBench out-domain [6], Hermes function call, Hermes function call single-turn, Hermes JSON mode single-turn.
- `text_long_qa`: ConvFinQA dev and Leval multidoc QA.
- `text_hallucination_factuality`: HEB RAG hallucination QA.

The evidence base used in this paper is summarized below.

Evidence block	Source scope	Used for
Remote ORP archive	P1 sweep over 1,319 GSM8K examples; full GSM8K matrix; P1/P2/P3 and e-value analyses	Motivation for non-destructive AG, intervention positions, e sensitivity, and fix/harm gating
Qwen3.5-9B aligned mining	8 text/tool datasets, old-P + new-P candidates, 241,056 feature rows	Main function-call/tool-use offline gate evidence
Qwen3-8B aligned mining	8 text/tool datasets, old-P + new-P candidates, 165,432 feature rows	Cross-architecture JSON/prefix-format evidence
Qwen2.5-7B aligned mining	8 text/tool datasets, BS64 old-P + new-P candidates, 241,056 feature rows	Boundary case for oracle headroom without gate transfer
Feature ablation and bootstrap CI	Aligned meta CSV pools, no-leak/diagnostic exclusions, stable hash split, 7,396 samples per model	Confidence intervals and feature-family ablation for tool-route gates

Evidence block	Source scope	Used for
Qwen3.5-9B MC/choice diagnostics	Aligned rerun over C-Eval and MMLU-Pro raw detail JSONs with the stable hash split	Negative diagnostic evidence against claiming universal choice-task gains
Divergence/layer trace diagnostics	Aligned text pools and Qwen3.5 choice trace details under the same stable split	Mechanistic evidence for trajectory divergence and limits of layer-trace gating
Qwen3 vs Qwen2.5 comparison	Full-attention family gate comparison	Evidence that shared feature families do not imply transferable gates

4.4 Gate protocol and leakage controls

Coverage alone is not enough. A broad candidate sweep can easily produce misleading conclusions if the final result is selected after seeing labels. We therefore impose leakage controls before reporting results.

First, every run is compared against an explicit baseline for the same model, dataset, prompt/evaluator, and decoding setup. This prevents an AG result from being compared against a stale or mismatched baseline.

Second, candidate generation and gate evaluation are separated. The candidate matrix is allowed to be broad for discovery, but deployable claims must come from either a fixed single configuration or a learned gate using features that would be available online.

Third, learned gates use only gate-policy features. Diagnostic-only and post-hoc fields are ranked separately for explanation and are never used for the main learned-gate results. Mixed feature combinations that require additional inference passes, extra judge calls, incompatible prompt/parser assumptions, or severe latency/TPS loss are excluded from deployable gate claims.

Fourth, the design includes task-category and cross-model stress tests. A result that holds only under a pooled all-dataset view is not considered sufficient. The method must show where it works, where it weakly works, and where current gates fail.

The current learned-gate results should be read as deployable-feature- constrained offline evidence, not as a completed prospective online A/B validation. Independent splits, time splits, and new held-out datasets remain necessary before stronger deployment claims can be made.

4.5 Accounting levels and gate terminology

We distinguish four evaluation levels.

Level	Meaning	Deployment relevance
Combination oracle	Per sample, choose the best non-baseline AG candidate after observing scores	Upper bound only
Best fixed single configuration	One fixed AG run key chosen for a dataset/category, then applied uniformly	Approximate fixed-route evidence; not a per-sample oracle
Learned gate	Learned policy chooses among candidates using online-observable features	Main deployable-feature-constrained offline evidence
Diagnostic-only analysis	Uses post-hoc or label-adjacent signals	Explanation only

We use the following gate names in the result tables. A **mixed gate** is a category-level learned ranker that combines multiple online-observable feature families such as prefix/format features, early logprob trajectory, uncertainty, and divergence/energy features. A **ridge gate** is a regularized linear utility ranker over the same deployable feature pool. A **single indicator** gate uses one online-safe feature or indicator. These gates are offline evaluations of deployable feature policies; they are not prospective online A/B results.

The standard term in this paper is **combination oracle**. When the candidate set is described as the union of several position subsets, we sometimes refer to the same upper-bound construction as a union over those candidates; it is still a combination oracle and remains non-deployable because labels are observed before choosing the candidate.

We explicitly avoid three forms of overstatement.

Pooled multi-config bias occurs when many AG configurations are mixed and a sample is counted as captured if any configuration fixes it. This is useful for finding headroom but does not correspond to a single online policy.

Best-selected bias occurs when multiple accepted candidates exist and the correct one is chosen after the fact. This is oracle behavior, not deployment.

Row-level bias occurs when sample-by-configuration rows are counted as if they were independent questions. We report sample-level results for deployment claims.

4.6 Metrics: native score, strict fix/harm, and partial10 fix/harm

The main result tables use each dataset's native score for baseline, current, and oracle values. ToolBench-style tasks are strict binary tasks: the output either satisfies the tool-call criterion or not. For these tasks, strict fix/harm and accuracy are appropriate. Partial-credit tasks such as function-call F1, long QA, or RAG factuality require additional care. A strict `score == 1` metric can understate baseline quality and distort AG gains when many examples receive nonzero partial scores.

We define strict fix/harm from the full-credit indicator. A sample is a strict fix if the baseline is not full-credit and the AG candidate is full-credit; it is a strict harm if the baseline is full-credit and the AG candidate is not full-credit. We therefore report strict fix/harm as a safety diagnostic, not as the only score for partial-credit datasets. We also use partial10 fix/harm as a coarser partial-credit safety signal. For a native score s in $[0, 1]$, define $b(s) = \text{floor}(10s)$ clipped to the integer range $[0, 9]$ for $s < 1$, and set $b(1) = 10$. A sample is a partial10 fix if $b(s_{\text{AG}}) > b(s_{\text{base}})$ and a partial10 harm if $b(s_{\text{AG}}) < b(s_{\text{base}})$. Thus partial10 measures movement between ten coarse score intervals, with exact full credit kept as a separate endpoint, rather than requiring a full strict transition from incorrect to correct. Strict fix/harm is used as the conservative safety diagnostic; partial10 is used to detect aggregate benefit or degradation on datasets whose evaluators provide meaningful partial credit.

The aggregate strict-score check at the start of Section 5 is only a sanity check over candidate-space signal; the main claims use route-specific native scores, oracle capture, and fix/harm diagnostics.

4.7 Feature policy and evidence boundary

The mining reports separate gate-policy features from diagnostic-only features.

Feature class	Examples	Used by learned gate?	Role
Prefix/format/schema features	JSON node count, max depth, format score, argument-name counts	Yes	Online structural evidence for tool/function/JSON routes
Early trajectory features	early logprob, acceptance slope, first/second/third-token deltas	Yes	Online confidence and stability signal
Baseline uncertainty features	top-k probability gaps, baseline acceptance statistics	Yes	Online uncertainty and fallback signal
Divergence/energy features	first divergence, local divergence ratios, retained energy, keep dimension	Yes when computable online	Candidate/baseline separation and FFN-state diagnostics
Post-fork/post-generation diagnostics	cross-PPL on generated AG tokens, post5 overlap, change-energy fields	No	Explanation only; not used in main deployable gates
Label-derived or correctness features	answer score, evaluator outcome, post-hoc best candidate	No	Forbidden for deployable claims

This policy is intentionally conservative. It lets the paper report oracle headroom, learned-gate capture, and diagnostic explanations without collapsing them into the same evidence level. The strongest current evidence is therefore not "AG always helps"; it is that, under a leakage-controlled offline protocol, model- and category-specific gates capture useful AG candidate space on selected tool-structured routes.

5. Experiment Results: Cross-Model and Cross-Task AG Effects

Before reporting route-specific results, we use an aggregate strict-score view only as a sanity check. Across 9,886 examples, the baseline reaches 981 full-score answers (9.92%); the best single P3/e0.93 configuration reaches 1,091 (11.03%, +1.10 pp); and a three-position combination oracle over the union of P1/P2/P3 candidates reaches 1,159 (11.72%, +1.80 pp), with 178 full-score fixes and 141 harms. This confirms that AG has nonzero candidate-space signal, but it also shows why pooled strict accounting is too coarse for deployment: the harm rate remains large, and the metric mixes binary and partial-credit tasks. We therefore treat this aggregate view as a diagnostic and make the main claims only through the model- and task-specific results below.

The decode-only design also shapes how the results should be interpreted. AG does not modify prefill states; it starts only when the model begins generating tokens. This makes structured generation, function calling, and JSON emission natural targets because there is a multi-token output trajectory in which a small FFN shift can change a branch. By contrast, multiple-choice and other single-label choice settings often expose only a one-token decision, so weak MC gains are expected and should not be read as evidence that the FFN intervention has no trajectory-level effect.

5.1 Qwen3.5-9B: tool/function-call is strongest

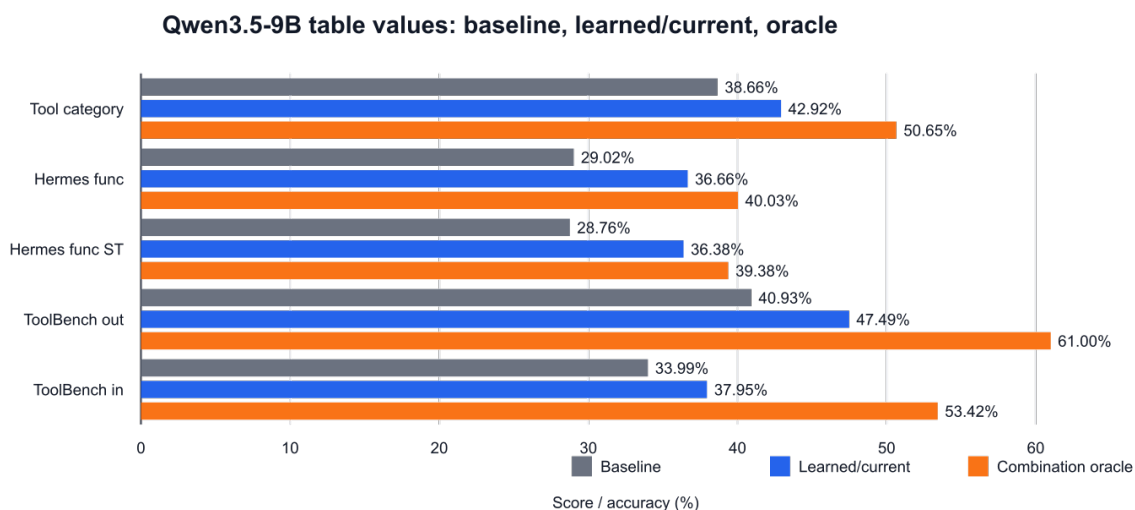


Figure 6. Qwen3.5-9B tool-structured results: baseline, learned gate, and oracle score by route.

Qwen3.5-9B shows the clearest deployment signal in tool-structured tasks.

Category/method	Baseline	Learned/current	Lift	Oracle delta	Oracle capture
Tool/structured, category mixed gate	38.66%	42.92%	+4.27 pp	+11.99 pp	35.6%
Tool/structured, category ridge gate	38.66%	42.87%	+4.21 pp	+11.99 pp	35.1%
Tool/structured, best fixed single config	38.66%	41.12%	+2.46 pp	+11.99 pp	20.5%

The corresponding combination-oracle upper bound is 50.65%, i.e. +11.99 pp over baseline. It is reported as candidate-space headroom rather than as a deployable method.

The same category-level table must be read with fix/harm accounting. The Qwen3.5-9B mixed gate has a positive strict balance (388 fixes / 317 harms) and a stronger partial-credit balance (2,425 / 1,401). The combination oracle is larger but not deployable; it reaches 838 / 349 under strict accounting and 4,125 / 1,251 under partial10 accounting. These numbers support the claim that the learned gate captures real signal, while still leaving substantial unclaimed oracle space.

At the dataset level, Hermes function-call tasks are the strongest current online-gate evidence.

Dataset	Baseline	Combination oracle	Best learned	Learned lift	Oracle capture
Hermes function call	29.02%	40.03%	36.66%	+7.64 pp	69.4%
Hermes function call single-turn	28.76%	39.38%	36.38%	+7.62 pp	71.7%
ToolBench out-domain	40.93%	61.00%	47.49%	+6.56 pp	32.7%
ToolBench in-domain	33.99%	53.42%	37.95%	+3.96 pp	20.4%

These results show that Qwen3.5-9B is not merely helped by one globally superior AG configuration. The learned gate improves substantially over the best fixed single configuration in the category aggregate, indicating that sample-level observable routing carries real information.

Qwen3.5-9B also shows positive but weaker results outside tool-structured tasks. HEB RAG factuality improves from 21.64% to 25.10% under the best category gate (+3.47 pp), but this category currently contains only one dataset and should not be generalized broadly. Long QA is weakly positive: 3.71% to 4.42% under a mixed category gate (+0.71 pp), with small absolute gains.

The fixed-configuration audit gives a more granular view of where NS AG helps. For Qwen3.5-9B, the best fixed non-NS entropy candidate wins six datasets, the best fixed NS entropy candidate wins five datasets, and one dataset is tied. The MC diagnostic rows include C-Eval [10], GSM8K [8], HLE proxy [11], and MMLU-Pro [9]. This is why the paper treats NS as a useful candidate family rather than a globally dominant replacement for non-NS AG.

Dataset	Category	Baseline	Best non-NS fixed	Lift	Best NS fixed	Lift	Winner
C-Eval	MC diagnostics	79.27%	79.49%	+0.22 pp	79.35%	+0.07 pp	non-NS
GSM8K	MC diagnostics	89.39%	89.54%	+0.15 pp	88.78%	-0.61 pp	non-NS
HLE proxy	MC diagnostics	5.08%	5.32%	+0.24 pp	4.96%	-0.12 pp	non-NS

Dataset	Category	Baseline	Best non-NS fixed	Lift	Best NS fixed	Lift	Winner
MMLU-Pro	MC diagnostics	53.48%	53.51%	+0.02 pp	53.56%	+0.07 pp	NS
HEB RAG hallucination QA	Factuality	22.24%	22.15%	-0.09 pp	23.75%	+1.51 pp	NS
ConvFinQA dev	Long QA	1.28%	1.88%	+0.60 pp	2.21%	+0.94 pp	NS
Leval multidoc QA	Long QA	31.94%	31.60%	-0.34 pp	32.14%	+0.20 pp	NS
Hermes function call	Tool/structured	28.89%	31.32%	+2.43 pp	30.30%	+1.41 pp	non-NS
Hermes function call single-turn	Tool/structured	28.89%	31.57%	+2.68 pp	30.34%	+1.45 pp	non-NS
Hermes JSON mode single-turn	Tool/structured	72.43%	72.50%	+0.07 pp	72.55%	+0.12 pp	NS
ToolBench in-domain	Tool/structured	34.57%	35.96%	+1.39 pp	35.96%	+1.39 pp	tie
ToolBench out-domain	Tool/structured	43.15%	46.22%	+3.07 pp	45.58%	+2.43 pp	non-NS

The fixed rows also expose a matched P-by-e result inside the Qwen3.5-9B tool reports. These tables reuse the pre-specified e0.93/e0.95 retained-mass grid from Section 4.2; they do not derive a new e-value from the reported results. In the non-NS entropy slice, Hermes function call is best at P1s/e0.93 and remains positive at P1s/e0.95, while P2a and P2b are negative. ToolBench out-domain is different: P2b/e0.93 is the strongest fixed candidate, P2b/e0.95 remains positive but weaker, and P2a changes sign across the two e-values. Sections 5.2--5.3 report the corresponding matched new-P slices for Qwen3-8B and Qwen2.5-7B; Section 6.1 compares the cross-model pattern and interprets why this position/e interaction matters.

Fixed non-NS slice	Best P/e cell	Same-position e comparison	Cross-position contrast
Hermes function call	P1s/e0.93: 31.32% (+2.43 pp)	P1s/e0.95: 30.31% (+1.42 pp)	P2a and P2b are negative at both inspected e-values
ToolBench out-domain	P2b/e0.93: 46.22% (+3.07 pp)	P2b/e0.95: 44.56% (+1.41 pp)	P1s is positive but weaker; P2a changes sign with e

5.2 Qwen3-8B: JSON/prefix-format is strongest

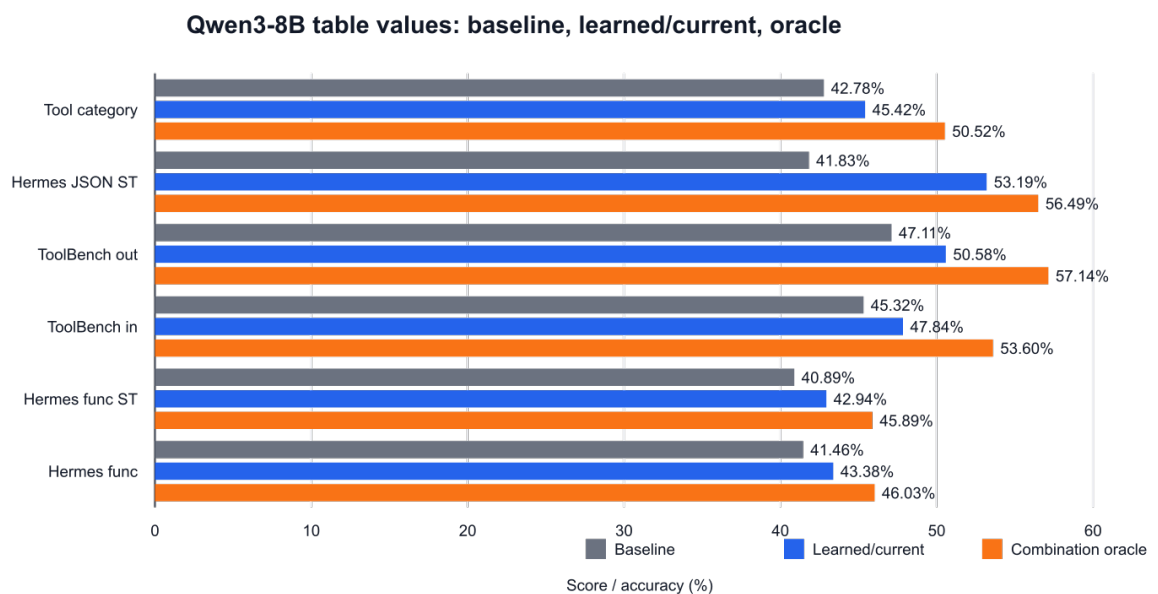


Figure 7. Qwen3-8B tool-structured results: baseline, learned gate, and oracle score by route.

Qwen3-8B also shows positive AG signal in tool-structured tasks, but the strongest subtask changes.

Category/method	Baseline	Learned/current	Lift	Oracle delta	Oracle capture
Tool/structured, category mixed gate	42.78%	45.42%	+2.64 pp	+7.77 pp	34.0%
Tool/structured, category ridge gate	42.74%	45.23%	+2.49 pp	+7.77 pp	32.1%
Tool/structured, single indicator	42.75%	45.34%	+2.59 pp	+7.77 pp	33.3%

The corresponding combination-oracle upper bound is 50.52%, i.e. +7.77 pp over the category baseline. It is used only to define candidate-space headroom and oracle capture.

Fix/harm accounting shows a different risk profile from Qwen3.5-9B. The Qwen3-8B mixed gate has a small negative strict balance in the pooled tool category (22 / 27), while its partial10 balance is strongly positive (218 / 40). This is why the Qwen3-8B claim is framed around structured-output score improvement rather than zero-harm strict binary deployment.

At dataset level, Qwen3-8B's strongest result is Hermes JSON mode.

Dataset	Baseline	Combination oracle	Best learned	Learned lift	Oracle capture
Hermes JSON mode single-turn	41.83%	56.49%	53.19%	+11.36 pp	77.6%
ToolBench out-domain	47.11%	57.14%	50.58%	+3.47 pp	36.0%
ToolBench in-domain	45.32%	53.60%	47.84%	+2.52 pp	29.2%
Hermes function call single-turn	40.89%	45.89%	42.94%	+2.05 pp	41.0%
Hermes function call	41.46%	46.03%	43.38%	+1.92 pp	42.1%

This contrasts with Qwen3.5-9B. Qwen3-8B's strongest signal is prefix/format and JSON structure; Qwen3.5-9B's strongest signal is function-call/tool structured output. Thus AG appears across both models, but the best gate is not universal.

The fixed-report pools also show that Qwen3-8B does not reduce to a single AG family. NS is stronger on ToolBench, while non-NS remains stronger on Hermes function-call and JSON-mode rows. The fixed rows below are reported as best-within-family comparisons; old-P and new-P pools are not always identical, so the lift attached to each score is its paired fixed-pool lift.

Dataset	Category	Best non-NS fixed (lift)	Best NS fixed (lift)	Winner
HEB RAG hallucination QA	Factuality	32.55% (+0.24 pp)	32.55% (+0.24 pp)	tie
ConvFinQA dev	Long QA	5.97% (+0.60 pp)	6.05% (+0.20 pp)	NS
Leval multidoc QA	Long QA	27.58% (+0.41 pp)	27.55% (+0.38 pp)	non-NS
Hermes function call	Tool/structured	57.63% (+0.60 pp)	57.50% (+0.47 pp)	non-NS
Hermes function call single-turn	Tool/structured	57.93% (+1.22 pp)	57.65% (+0.95 pp)	non-NS
Hermes JSON mode single-turn	Tool/structured	41.48% (+0.77 pp)	39.60% (-1.11 pp)	non-NS
ToolBench in-domain	Tool/structured	45.47% (+0.88 pp)	46.91% (+2.33 pp)	NS
ToolBench out-domain	Tool/structured	53.52% (+1.76 pp)	57.42% (+5.66 pp)	NS

The matched new-P non-NS slice shows that Qwen3-8B has a different P/e profile from Qwen3.5-9B. On Hermes function call, P2a is the positive route and P1s/P2b are negative; on ToolBench out-domain, all three refined points are positive at e0.93, with P2b/e0.93 the strongest non-NS cell.

Qwen3-8B fixed non-NS slice	Strongest P/e cell	Same-position e comparison	Cross-position contrast
Hermes function call	P2a/e0.93: 57.63% (+0.60 pp)	P2a/e0.95: 57.57% (+0.55 pp)	P1s and P2b are negative at both inspected e-values
ToolBench out-domain	P2b/e0.93: 53.52% (+1.76 pp)	P2b/e0.95: 52.73% (+0.98 pp)	P1s and P2a are also positive at e0.93

5.3 Qwen2.5-7B: oracle headroom without deployable gate

Qwen2.5-7B table values: baseline, learned/current, oracle

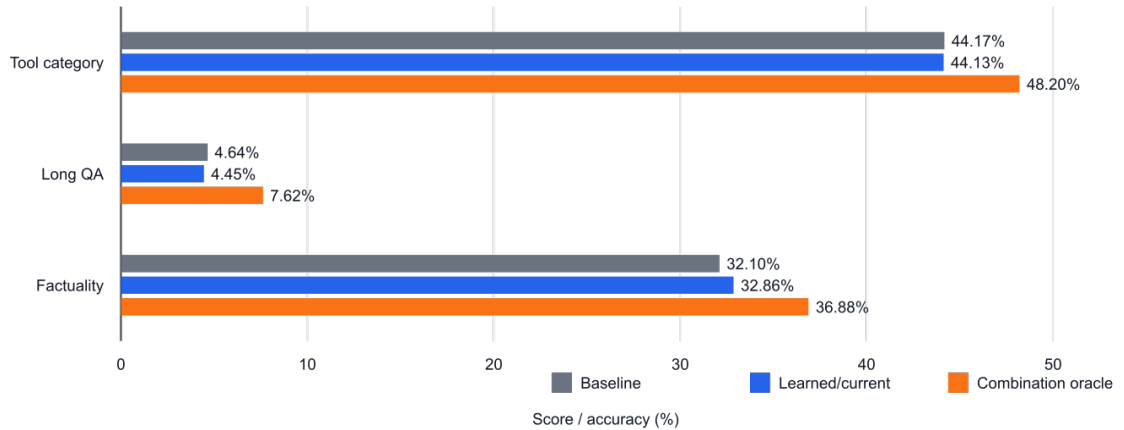


Figure 8. Qwen2.5-7B boundary result: baseline, learned/current, and oracle score show headroom without a valid route.

Qwen2.5-7B provides an important negative or boundary result. Its AG candidate space contains oracle headroom, but current learned gates do not capture it at the category level.

Category	Best learned method	Baseline	Learned/current	Learned lift	Oracle delta	Oracle capture
Tool/structured	mixed gate	44.17%	44.13%	-0.04 pp	+4.03 pp	-0.94%
Long QA	ridge gate	4.64%	4.45%	-0.19 pp	+2.98 pp	-6.30%
Factuality	mixed gate	32.10%	32.86%	+0.76 pp	+4.78 pp	15.87%

This result prevents overclaiming. The presence of combination-oracle headroom does not by itself justify deployment. A model can contain useful AG alternatives while lacking a current online feature policy that selects them safely. For Qwen2.5-7B, the best positive dataset-level result is ToolBench out-domain (+1.54 pp learned lift), but the overall tool category gate is not yet valid.

The fixed-configuration comparison reinforces this boundary-case reading. There are positive fixed candidates, especially NS candidates on Long QA and ToolBench, but they do not translate into a deployable positive category gate. As in Qwen3-8B, the fixed rows use paired fixed-pool lifts.

Dataset	Category	Best non-NS fixed (lift)	Best NS fixed (lift)	Winner
HEB RAG hallucination QA	Factuality	32.55% (+0.64 pp)	32.42% (+0.51 pp)	non-NS
ConvFinQA dev	Long QA	2.89% (+0.47 pp)	3.29% (+0.87 pp)	NS
Leval multidoc QA	Long QA	34.00% (+0.78 pp)	34.01% (+0.79 pp)	NS
Hermes function call	Tool/structured	42.99% (-0.11 pp)	42.98% (-0.13 pp)	non-NS
Hermes function call single-turn	Tool/structured	42.94% (-0.04 pp)	42.98% (-0.00 pp)	NS
Hermes JSON mode single-turn	Tool/structured	82.19% (+0.14 pp)	82.42% (+0.36 pp)	NS
ToolBench in-domain	Tool/structured	25.88% (+0.06 pp)	26.39% (+0.57 pp)	NS
ToolBench out-domain	Tool/structured	25.48% (+0.64 pp)	25.86% (+0.77 pp)	NS

The matched new-P non-NS slice explains part of the boundary result. For Qwen2.5-7B, Hermes function call is negative across the displayed refined positions and e-values. ToolBench out-domain has a small positive P1s route, but P2a and P2b are negative. This is consistent with the larger result: the candidate space contains some useful alternatives, yet the current fixed non-NS new-P cells do not provide a robust category-level route.

Qwen2.5-7B fixed non-NS slice	Strongest P/e cell	Same-position e comparison	Cross-position contrast
Hermes function call	P1s/e0.95: 42.76% (-0.30 pp)	P1s/e0.93: 42.70% (-0.36 pp)	All displayed new-P cells are negative
ToolBench out-domain	P1s/e0.95: 25.48% (+0.64 pp)	P1s/e0.93: 25.10% (+0.26 pp)	P2a and P2b are negative at both inspected e-values

The Qwen2.5-7B result is also important at the feature level. The same broad families that appear useful on Qwen3-8B and Qwen3.5-9B still appear in the mining output, including prefix/format features, early logprob or acceptance trajectory, divergence, and energy-retention statistics. However, their thresholds and signs do not produce a positive category-level gate. Shared feature families are therefore not sufficient evidence for gate transfer.

5.4 NS AG versus non-NS AG

The fixed-configuration reports show that NS AG is useful, but not uniformly dominant. The dataset-level tables in Sections 5.1--5.3 show the main pattern: NS helps most when the task benefits from a local, windowed score, while non-NS remains competitive or better on several tool-format rows.

The category-level fixed-pool comparison makes the same point in a compact form. The table explicitly lists both scores rather than only the winning best fixed single-configuration candidate.

Model	Category	Baseline	Score with non-NS AG	Score with NS AG	Winner	Winner lift
Qwen3.5-9B	Tool/structured	38.92%	40.76%	40.11%	non-NS	+1.83 pp
Qwen3.5-9B	Long QA	4.22%	4.64%	5.00%	NS	+0.78 pp
Qwen3.5-9B	Factuality	22.24%	22.15%	23.75%	NS	+1.51 pp
Qwen3-8B	Tool/structured	49.76%	49.40%	50.20%	NS	+0.44 pp
Qwen3-8B	Long QA	10.89%	10.41%	10.57%	NS	-0.32 pp
Qwen3-8B	Factuality	32.31%	32.55%	32.55%	tie	+0.24 pp
Qwen2.5-7B	Tool/structured	43.95%	43.90%	43.90%	near tie	-0.04 pp
Qwen2.5-7B	Long QA	5.38%	5.76%	6.06%	NS	+0.68 pp
Qwen2.5-7B	Factuality	31.91%	32.55%	32.42%	non-NS	+0.64 pp

Combination-oracle selections add a different view: NS candidates account for most oracle-selected rows in Qwen3-8B and Qwen2.5-7B category pools, often above 89% of selected samples. This does not prove that NS is the deployable choice, because the oracle sees labels and chooses per sample. It does show that windowed NS AG contributes substantial candidate-space diversity even when the best fixed or learned route is not NS.

6. Analysis: Mechanisms, Boundaries, and Gate Limits

6.1 Position and e-value sensitivity

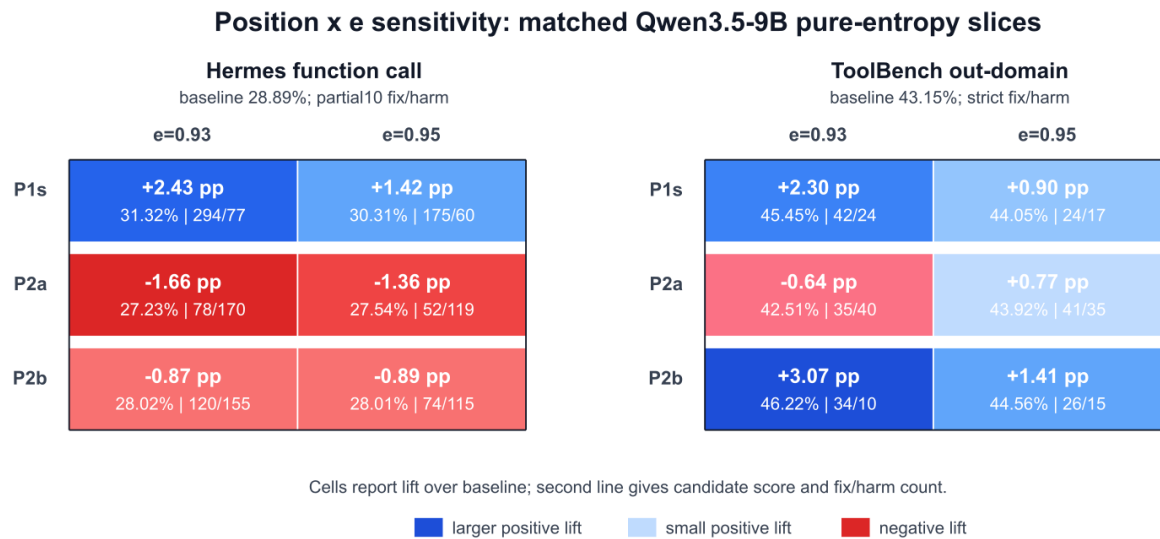


Figure 9. Position-by-e sensitivity: matched Qwen3.5-9B pure-entropy slices show that both the FFN position and e-value change the sign and size of AG effects.

Across old-P and new-P runs, AG behavior depends on both position and e. The GSM8K matrix shows that P1/P2/P3 and their combinations produce small and non-monotonic changes around a high baseline. To make the interaction explicit, Table 8 reports matched Qwen3.5-9B pure-entropy slices where the only displayed variables are the refined FFN position and the e-value. On Hermes function call, P1s is positive at both e-values, while P2a and P2b are negative at both e-values; lowering e from 0.95 to 0.93 increases the P1s gain but also changes the fix/harm balance.

Position	e=0.93 candidate	e=0.95 candidate	Local readout
P1s	31.32% (+2.43 pp; 294/77 partial10)	30.31% (+1.42 pp; 175/60 partial10)	Best route for function-call format repair
P2a	27.23% (-1.66 pp; 78/170 partial10)	27.54% (-1.36 pp; 52/119 partial10)	Negative at both e-values
P2b	28.02% (-0.87 pp; 120/155 partial10)	28.01% (-0.89 pp; 74/115 partial10)	Near-flat but still harmful

ToolBench out-domain gives a different P-by-e profile. P2b becomes the best single route, P1s remains positive but weaker, and P2a changes sign between the two inspected e-values. Thus, the same refined FFN point cannot be selected globally, and the same e-value does not have a uniform effect across positions.

Position	e=0.93 candidate	e=0.95 candidate	Local readout
P1s	45.45% (+2.30 pp; 42/24 strict)	44.05% (+0.90 pp; 24/17 strict)	Positive, but not the best ToolBench route
P2a	42.51% (-0.64 pp; 35/40 strict)	43.92% (+0.77 pp; 41/35 strict)	Sign changes with e
P2b	46.22% (+3.07 pp; 34/10 strict)	44.56% (+1.41 pp; 26/15 strict)	Best ToolBench out-domain route

The same matched new-P view across checkpoints shows that the interaction is not a Qwen3.5-only artifact. Table 10 summarizes the strongest fixed non-NS new-P cells for the same two routes across all three checkpoints. Qwen3-8B moves the Hermes function-call positive route from P1s to P2a, while Qwen2.5-7B has no positive displayed Hermes new-P cell. ToolBench out-domain is positive for Qwen3.5-9B and Qwen3-8B at P2b/e0.93, but Qwen2.5-7B's positive new-P cell is the smaller P1s/e0.95 route. Thus, position and e interact with both task and model.

Model	Hermes function call strongest new-P cell	ToolBench out-domain strongest new-P cell	Cross-model readout
Qwen3.5-9B	P1s/e0.93: 31.32% (+2.43 pp)	P2b/e0.93: 46.22% (+3.07 pp)	Strongest tool/function-call evidence; route changes by task
Qwen3-8B	P2a/e0.93: 57.63% (+0.60 pp)	P2b/e0.93: 53.52% (+1.76 pp)	Positive cells exist, but the best function-call position shifts
Qwen2.5-7B	P1s/e0.95: 42.76% (-0.30 pp)	P1s/e0.95: 25.48% (+0.64 pp)	Boundary case: small or negative fixed new-P gains

The refined positions therefore have task-specific and model-specific value, but the value is not uniform. P1s is strongest for Qwen3.5-9B function-call reports; P2a is the positive Qwen3-8B function-call route; P2b is useful on ToolBench out-domain for Qwen3.5-9B and Qwen3-8B; and Qwen2.5-7B mostly shows boundary behavior in this matched fixed slice. GSM8K reinforces the same caution: even the best fixed MC configuration is only +0.15 pp, with nearly balanced fix/harm.

The correct conclusion is therefore not "one position wins everywhere." The evidence supports a structured candidate system in which position, e, algorithm variant, model, and task category jointly determine whether AG helps.

6.2 Interpreting NS versus non-NS AG

The comparison in Section 5.4 changes the interpretation of NS AG. The right conclusion is not that NS AG dominates non-NS AG, nor that non-NS entropy AG is sufficient. The evidence supports a two-family candidate system. Non-NS AG is the sharper current-token amplitude family; NS AG is the windowed local-trajectory family. Which family is preferable depends on the model, task category, and whether the task rewards current-token specificity or short-horizon stability.

The first distinction is between an intra-family win and a deployable gain over baseline. In Qwen3.5-9B, the category-level fixed-pool comparison is positive and interpretable: tool/structured favors non-NS (40.76% versus 40.11%), while Long QA and factuality favor NS (5.00% versus 4.64%, and 23.75% versus 22.15%). This supports a clean mechanism hypothesis: function-call and tool-format rows often benefit from sharp amplitude decisions at specific FFN positions, whereas long QA and factuality benefit more from a short window that stabilizes the dimension ranking.

Qwen3-8B complicates the story. Its tool/structured category favors NS in the fixed-pool comparison (50.20% versus 49.40%), but the strongest learned result in Section 5.2 is Hermes JSON mode, where format/prefix features dominate the online gate. Its Long QA row also favors NS over non-NS (10.57% versus 10.41%), but both are below the paired baseline of 10.89%. Thus an NS-over-non-NS win is not automatically a deployment win. It means NS is the better member of the fixed AG candidate family for that pool, not that a fixed AG route should be enabled without a gate.

Qwen2.5-7B is the boundary case. Long QA again favors NS (6.06% versus 5.76%), but factuality favors non-NS (32.55% versus 32.42%), and tool/structured is a near tie below baseline. This shows that the NS/non-NS choice is not explained by category alone. Architecture, model calibration, feature thresholds, and the available online gate all matter. A family can be useful in the oracle or fixed-pool view while still failing to produce a safe deployable category gate.

The three checkpoints differ in ways that plausibly explain this interaction, although the current experiments should be read as mechanistic evidence rather than causal proof. Qwen3.5-9B uses a hybrid text stack with many linear-attention layers and periodic full-attention layers. Qwen3-8B and Qwen2.5-7B are dense full-attention models, but they differ in depth and FFN expansion: Qwen3-8B has 36 layers with a 4096-to-12288 SwiGLU FFN, while Qwen2.5-7B has 28 layers with a 3584-to-18944 SwiGLU FFN. Since AG acts on FFN activation amplitudes, these differences can change both the sharpness of current-token peaks and the stability of short-window activation trajectories.

This gives a model-level interpretation of the fixed-pool results. In Qwen3.5-9B, the hybrid attention stack may make local FFN trajectories useful for long-QA and factuality rows, where NS wins, while function-call rows still benefit from sharper current-token amplitude decisions, where non-NS wins. In Qwen3-8B, dense full attention and strong prefix/format observables shift the tool signal: NS is better in the category fixed pool, but the deployable- feature-constrained learned gain is driven by JSON/prefix features rather than by a universal NS rule. In Qwen2.5-7B, the much wider FFN expansion provides candidate-space headroom, but the gate does not reliably convert that headroom into a positive category gate. The observed behavior is therefore best understood as a model-by-task interaction over FFN trajectories, not as an intrinsic ranking of NS above non-NS or vice versa.

Mechanistically, non-NS AG ranks dimensions using the instantaneous activation or entropy signal at the current decoding point. It is therefore well matched to tasks where the correct continuation depends on a crisp local format decision: opening or closing JSON structure, selecting a function-call slot, or committing to a tool argument. In those cases, smoothing the score over a window can blur a useful spike. This explains why Qwen3.5-9B function-call rows and Qwen3-8B JSON rows can prefer non-NS in the dataset-level tables.

NS AG changes the same mask interface by changing the score used to rank dimensions. With w_4 , the Newton-Schulz score is computed from a short window of recent activation states, so the selected dimensions reflect local trajectory rather than only the current vector. This is useful when the desired behavior is not a single sharp format decision but a more stable local semantic or evidence trajectory, as suggested by the Qwen3.5-9B Long QA and factuality rows and by the Qwen2.5-7B Long QA fixed-pool result.

The role of the window follows from this mechanism. A single-vector NS score is close to a normalized magnitude transform and adds limited ordering information. The windowed variant makes NS distinct by giving the ranker a short local matrix over recent decode activations. Larger windows may improve stability but can also dilute current-token specificity; smaller windows preserve specificity but may collapse toward non-NS behavior. The current evidence uses w_4 as a fixed design point, so the window-size claim should remain conservative.

Overall, Section 5.4 supports a routing conclusion. NS AG should be kept because it contributes distinct candidate-space diversity and wins important long-QA and factuality comparisons. Non-NS AG should also be kept because it remains stronger on several tool-format and current-token-sensitive rows. A robust AG system should expose both families and let a model- and task-aware gate decide when the extra local stability of NS is worth trading off against the sharper instantaneous response of non-NS.

6.3 Negative diagnostics and claim boundary

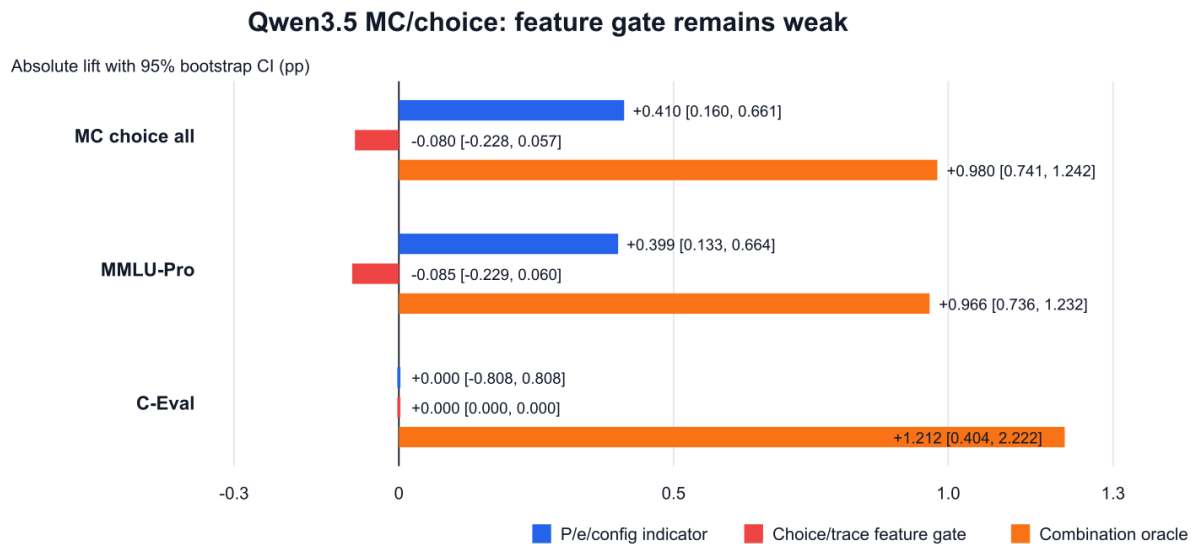


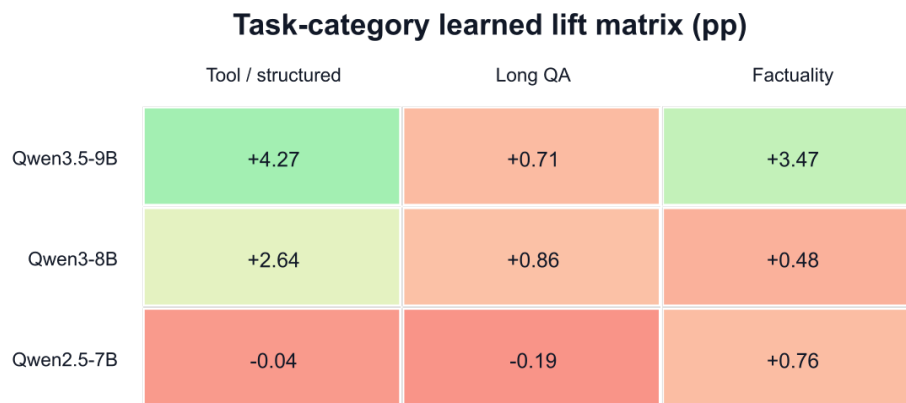
Figure 10. MC/choice diagnostics define the claim boundary: static P/e signals are small, and the current choice/trace feature gate is not positive.

The MC track is useful because it constrains the claim. The current aligned MC/choice rerun covers C-Eval and MMLU-Pro, built from raw detail JSONs with baseline/candidate alignment and the same stable split policy used by the aligned text/tool reruns. We do not use the older first-batch reports as the source for this conclusion. The rerun slightly narrows the old negative boundary: static P/e/config signals can be positive, yet the current choice/trace feature ranker is not a deployable positive gate.

Scope	Baseline	P/e/config indicator	Choice/trace feature gate	Combination oracle	Interpretation
MC choice all	54.27%	+0.410 pp, CI [0.160, 0.661]	-0.080 pp, CI [-0.228, 0.057]	+0.980 pp, CI [0.741, 1.242]	Small static signal; feature gate not established
MMLU-Pro	52.85%	+0.399 pp, CI [0.133, 0.664]	-0.085 pp, CI [-0.229, 0.060]	+0.966 pp, CI [0.736, 1.232]	Oracle headroom remains small
C-Eval	77.98%	+0.000 pp, CI [-0.808, 0.808]	+0.000 pp, CI [0.000, 0.000]	+1.212 pp, CI [0.404, 2.222]	High baseline; no learned choice trace signal

This negative evidence is important for the paper's scope. AG is not presented as a general-purpose accuracy booster for all text tasks. It is a gated candidate-generation mechanism whose current strongest offline gate evidence is tool-structured inference. It is also consistent with the decode-only intervention design: one-token choice tasks give AG little room to create or repair a downstream trajectory after the prompt prefill, whereas tool and JSON tasks expose multi-token action paths where an early branch can matter.

6.4 Why tool-structured tasks benefit most



Color encodes relative lift magnitude; greener cells are stronger, redder cells are weaker or negative.

Figure 11. Task-category learned lift matrix: tool-structured routes are the most consistent positive region.

Tool-structured tasks have two properties that make them suitable for AG.

First, the output space is constrained. Function calls, JSON objects, and tool arguments have detectable structure. Prefix and format features can observe whether the model is entering the right structural regime early in generation. This makes gate decisions easier than in open-ended long-form QA.

Second, small internal shifts can produce discrete external improvements. A slightly different FFN activation pattern may preserve the function name, argument order, or bracket structure needed by the evaluator. In long-form generation, by contrast, semantic correctness is diffuse and often depends on global reasoning, retrieval grounding, or multi-hop consistency. AG still has oracle headroom there, but current online gates capture less of it.

The sample-level audit makes this concrete. The table below shows three representative ToolBench out-domain examples from the Qwen3.5-9B `pure_entropy_ag|p1|e0.93` run, using first-action exact match rather than the older loose `contains` proxy. The positive examples are not semantic rewrites of the answer; they are format and action-selection repairs. The negative example shows the boundary: AG can also add a spurious chat suffix to an otherwise correct tool name.

Case	Task cue	Gold first action	Baseline first action	AG first action	Interpretation
Fix	quote categories	<code>list_categories_for_famous_quotes</code>	<code>list_categories_for_famous_quotesuser</code>	<code>list_categories_for_famous_quotes</code>	AG removed a prompt-boundary suffix that invalidated the action
Fix	streaming availability	<code>search_basic_free_for_streaming_availability</code>	<code>search_pro_for_streaming_availability</code>	<code>search_basic_free_for_streaming_availability</code>	AG shifted the selected tool from the pro endpoint to the basic/free endpoint required by the state
Harm	streaming availability	<code>search_pro_for_streaming_availability</code>	<code>search_pro_for_streaming_availability</code>	<code>search_pro_for_streaming_availabilityuser</code>	AG polluted a correct action with a boundary suffix, turning a clean hit into a failure

These cases also explain why the paper avoids relying on loose string-contains metrics for deployment claims. Some raw generations echo parts of the prompt or tool list; first-action parsing and sample-level fix/harm accounting are therefore necessary to distinguish real action repair from evaluator artifacts.

The aligned first-divergence rerun supports a trajectory-fork interpretation of these cases. In the tool-route test split derived from the aligned text/tool CSV pools, first-divergence features have clear bin-separation signal for Qwen3-8B and Qwen3.5-9B: Qwen3 reaches about 5.09 pp bin delta spread for both `first_divergence_step` and `first_divergence_norm`, while Qwen3.5 reaches about 4.81 pp and 4.56 pp respectively. Qwen2.5-7B is much weaker, with the largest divergence-feature spread about 1.46 pp. This pattern fits the cross-model results: trajectory divergence is informative where the gate works, but it is not a universal rule.

The corresponding layer-trace outcome aggregation is more cautious. On the current Qwen3.5 choice trace, fix and harm rows have very similar layer-trace means: rank-ratio 0.7042 versus 0.7015, residual magnitude 0.0734 versus 0.0727, and entropy 8.3561 versus 8.3982. These values make layer traces useful as mechanism diagnostics, not as a standalone gate. They also prevent the paper from claiming a causal low/mid/high-layer optimum or a universal early-token rule. The safest mechanism statement is therefore: AG can fork the generation trajectory at a discrete decision point, autoregressive decoding can propagate that fork, and current layer-level aggregates are not yet strong enough to serve as independent causal evidence.

6.5 Why cross-model results require routing

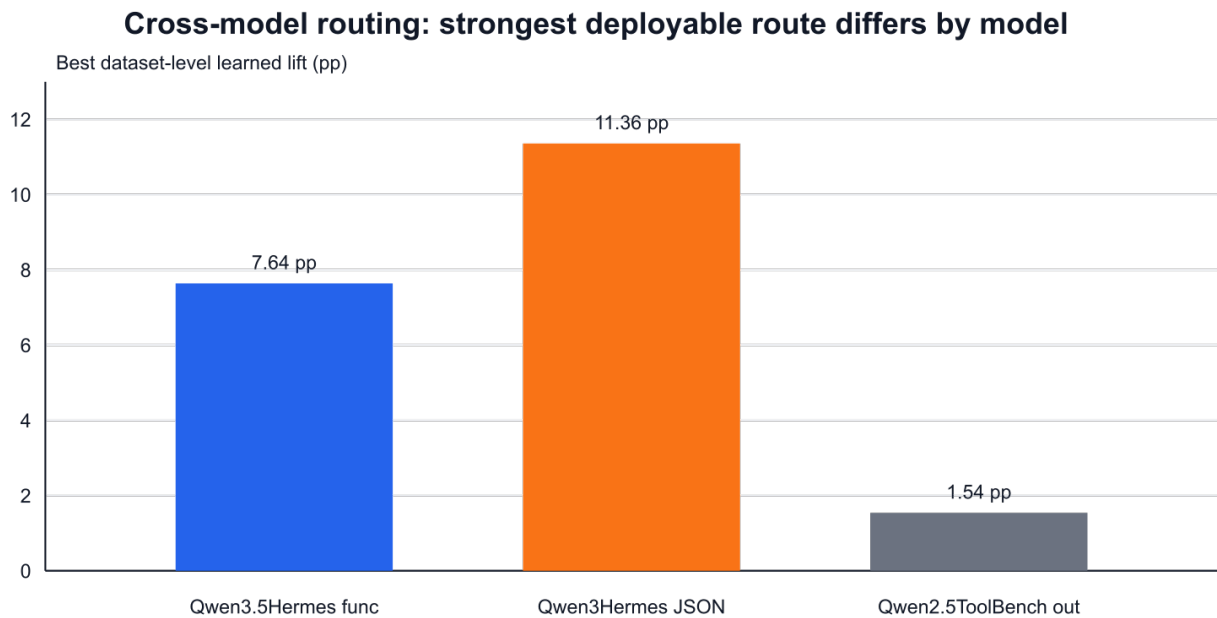


Figure 12. Cross-model routing: the strongest deployable route differs by checkpoint.

Qwen3.5-9B and Qwen3-8B both show positive tool-structured AG effects, but their strongest datasets and features differ. Qwen3.5-9B is strongest on function-call tasks. Qwen3-8B is strongest on JSON mode and prefix/format observables. Qwen2.5-7B shows that even within a broader Qwen family, another model can have oracle headroom without a working category-level gate.

The right deployment conclusion is therefore model- and category-specific routing. A universal AG switch is not supported. A more defensible design is:

1. choose model family and task category; 2. use a gate trained and validated for that route; 3. require held-out validation before enabling the route online; 4. fall back to baseline whenever the gate is not confident.

6.6 Oracle headroom versus deployable gain

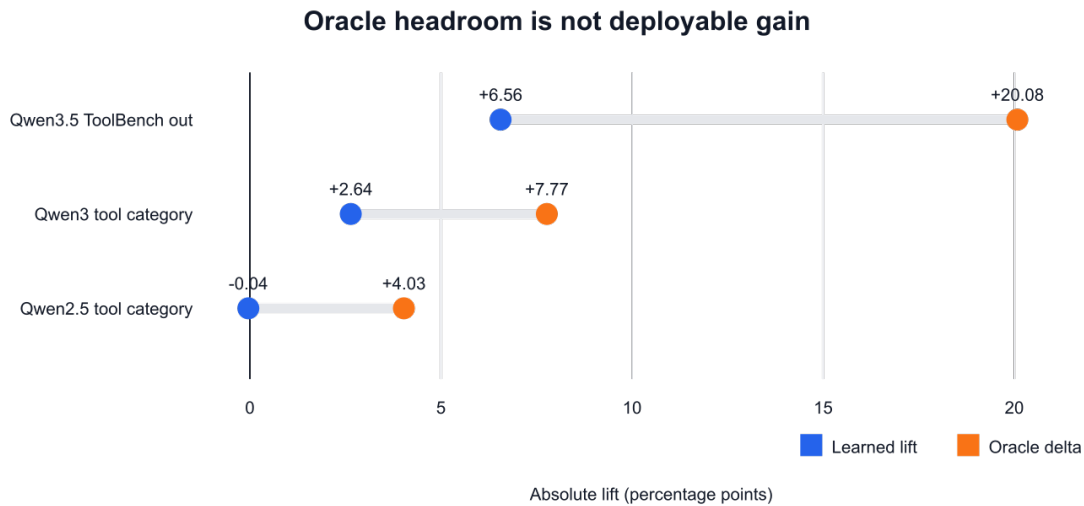


Figure 13. Oracle headroom versus deployable gain: large candidate-space headroom does not imply safe online capture.

Combination oracle is useful because it measures whether the AG candidate space contains helpful alternatives. It is not deployable because it chooses the right candidate after observing the answer score. A large oracle gap with a weak learned gate means the research problem has not been solved; it does not mean the method is ready.

ToolBench illustrates this point. Qwen3.5-9B ToolBench out-domain has a large combination oracle gain (+20.08 pp in dataset-level mining), but the best learned gate captures only part of it (+6.56 pp). Qwen2.5-7B is even more instructive: category-level tool oracle headroom exists (+4.03 pp), yet the learned gate is slightly negative. This is why our main claims emphasize learned-gate and category-specific results rather than only oracle space.

6.7 Feature families

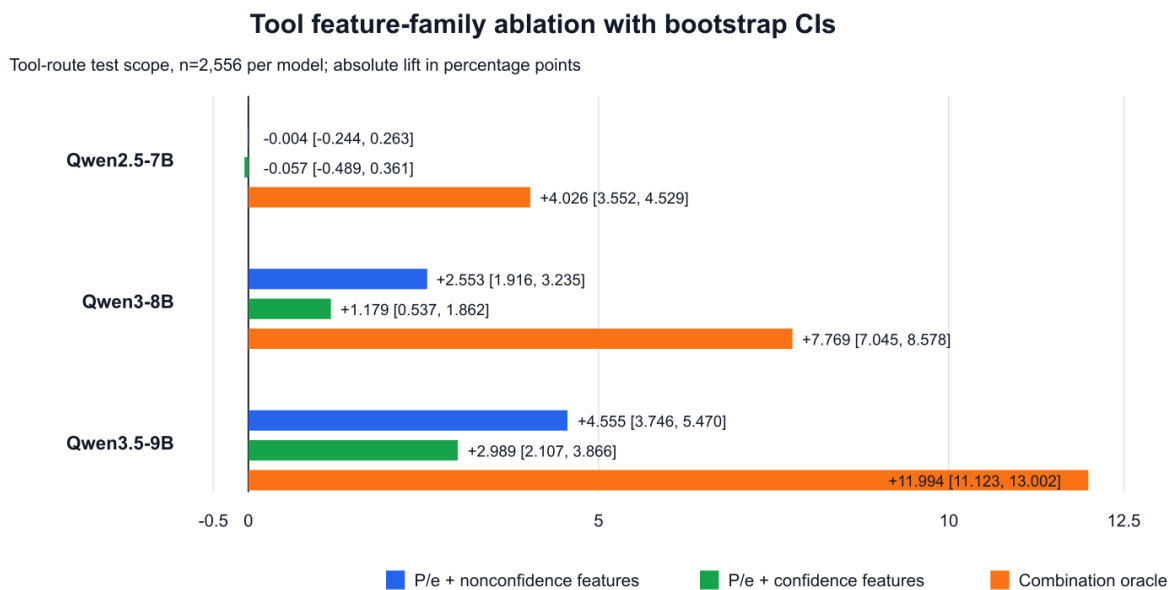


Figure 14. Feature-family ablation with bootstrap confidence intervals: Qwen3/Qwen3.5 support tool-route gates, while Qwen2.5 remains a boundary case.

The current strongest feature families are:

- Prefix/format/schema features, especially for JSON and function-call tasks.
- Early logprob and acceptance trajectory features, such as slope, trend, and first/second/third token changes.
- Divergence and first-divergence features, useful but not yet sufficient for long QA.

- Diagnostic post-fork features such as cross-PPL, which explain candidate plausibility but require care before online deployment.

The feature-family rerun provides the strongest statistical check in the current manuscript. It uses aligned meta CSV pools, no-leak/diagnostic exclusions, and the stable hash split. On the common tool-route test scope ($n=2,556$ per model), feature-family gates are CI-positive for Qwen3-8B and Qwen3.5-9B but not for Qwen2.5-7B.

Model	P/e/config only	P/e + nonconfidence features	P/e + confidence features	Combination oracle
Qwen2.5-7B	+0.039 pp, CI [-0.275, 0.327]	-0.004 pp, CI [-0.244, 0.263]	-0.057 pp, CI [-0.489, 0.361]	+4.026 pp, CI [3.552, 4.529]
Qwen3-8B	-0.111 pp, CI [-0.573, 0.363]	+2.553 pp, CI [1.916, 3.235]	+1.179 pp, CI [0.537, 1.862]	+7.769 pp, CI [7.045, 8.578]
Qwen3.5-9B	+1.756 pp, CI [1.101, 2.421]	+4.555 pp, CI [3.746, 5.470]	+2.989 pp, CI [2.107, 3.866]	+11.994 pp, CI [11.123, 13.002]

This table changes the feature conclusion in two ways. First, Qwen3/Qwen3.5 tool-route gains are not only point estimates; their bootstrap intervals are positive under the feature-family rerun. Second, Qwen2.5-7B still has oracle headroom but no CI-positive feature-family gate, so the boundary-case interpretation is strengthened rather than weakened.

At dataset level, the same pattern explains the model routes. Qwen3-8B Hermes JSON mode is carried by nonconfidence online features, including prefix/format and early trajectory fields: P/e plus nonconfidence features gives +11.449 pp, CI [9.374, 13.648]. Qwen3.5-9B Hermes function-call is similarly strong under P/e plus nonconfidence features (+7.514 pp, CI [6.580, 8.525]), while confidence-style baseline uncertainty features remain positive but smaller. This ablation groups prefix/format with other online nonconfidence features, so it should not be read as a pure prefix-only or prefix-excluded ablation. Instead, it shows that route-specific online feature families carry real signal after leakage controls.

6.8 Gate trade-off curves

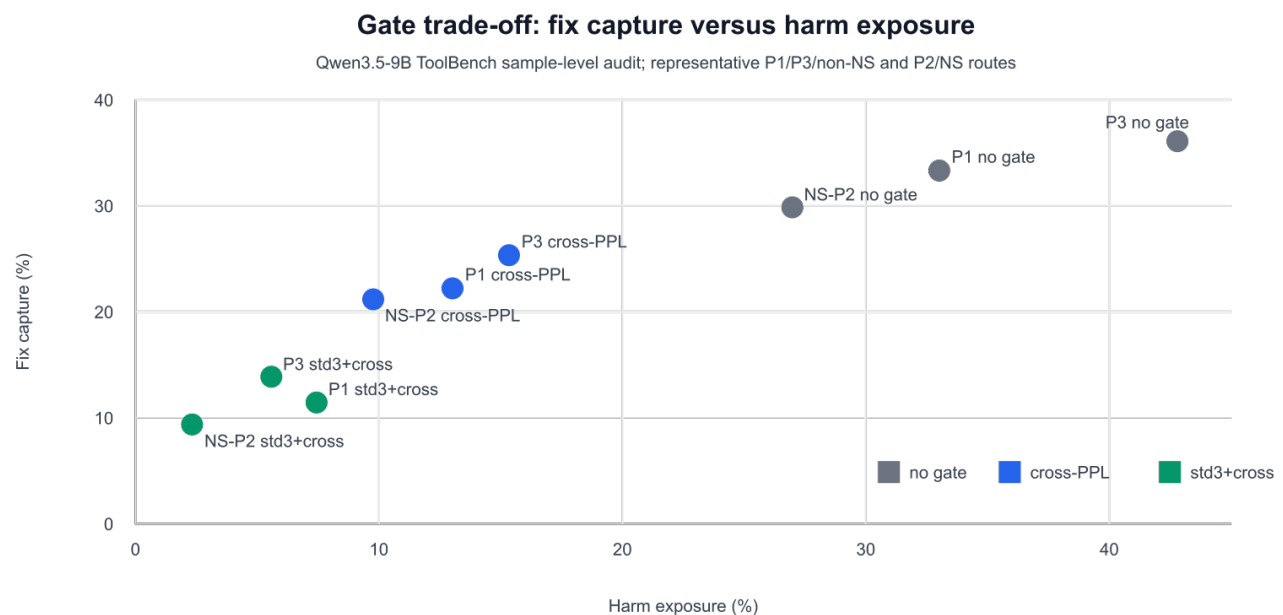


Figure 15. Gate trade-off curve: stricter gates reduce harm exposure while also giving up fix capture.

Gate choice is a Pareto trade-off rather than a binary question of whether AG helps. The Qwen3.5-9B ToolBench sample-level audit illustrates the pattern. For `pure_entropy_ag|p1|e0.93`, no gate captures 33.3% of available fixes but exposes 33.0% of harms; adding `crossppl <= 1.25833` reduces harm exposure to 13.0% while retaining 22.2% fix capture; adding the stricter `std3&crossppl` gate reduces harm exposure further to 7.4% but keeps only 11.5% fix capture. The same qualitative curve appears for `pure_entropy_ag|p3|e0.93` and `ns_entropy_ag|p2|e0.93|w4`. This is why the deployment discussion uses profiles such as safety-first, balanced, and performance-first rather than a single threshold.

7. Deployment: Safe Routes for Gated AG

The deployment implication is intentionally narrow: AG should first be considered for tool/function-call/structured agentic tasks under model routes where held-out learned-gate results are positive. In the current offline evidence base, Qwen3.5-9B function-call/tool routes and Qwen3-8B JSON structured-output routes are the strongest candidates for future held-out and online validation.

We recommend three deployment profiles.

Profile	Purpose	Gate style	Use when
Safety-first	Minimize harm	Conservative low-coverage gate and baseline fallback	High-risk domains
Balanced	Capture stable tool gains	Category/model-specific mixed or ridge gate	General tool calling
Performance-first	Capture more oracle space	Higher-coverage gate with monitoring	Internal or batch workflows

We do not recommend multi-flow multi-candidate voting as a first production architecture. It is useful offline, but online cost scales with the number of candidates. A more realistic path is a single-flow or lightly augmented baseline-plus-candidate design with online feature logging and fallback.

The current engineering evidence should be read as a deployment estimate, not as a serving benchmark. Hidden-dimensional points such as P1s and P3 require less activation storage than expansion-dimensional P2/P2a/P2b points, while offline multi-candidate mining is far more expensive than the intended online route. In the Qwen3.5-9B ToolBench audit, gate features such as early trajectory and cross-PPL materially reduce harm exposure, but post-fork cross-PPL is diagnostic-only in the present implementation because it depends on generated candidate tokens. A production version should therefore either use online approximations of those signals or measure the cost of a lightly augmented candidate path directly. We do not claim that AG is cost-free; the claim is narrower, namely that single-candidate hidden-dimensional routes are the plausible first deployment target, while broad multi-candidate voting belongs in offline mining.

Where possible, production systems should prefer hidden-dimensional points such as P1s or P3 over 4x-hidden points unless the expanded point produces clear task-specific value. P2/P2a/P2b remain scientifically important, but their deployment cost and noise profile are higher.

8. Limitations

This paper makes a deliberately bounded claim. The model evidence is limited to Qwen-family checkpoints. Qwen3.5-9B, Qwen3-8B, and Qwen2.5-7B cover useful attention and FFN-shape variation, but they do not establish behavior on other model families. The safest interpretation is therefore cross-checkpoint existence within one model family, not universal model transfer.

The task evidence is also concentrated. The strongest learned-gate gains appear on tool-structured and JSON/function-call routes. Long QA and factuality rows show oracle headroom and occasional fixed-family gains, but current gates do not yet support a broad open-domain deployment claim. Multiple-choice diagnostics are intentionally reported as weak or negative evidence.

Mechanistically, the P1s/P2a/P2b refinement remains only partially explained. Current reports show that refined SwiGLU points can help, but they do not yet prove that P1s cleanly isolates gate-branch correction or that P2a/P2b are orthogonal mechanism probes. Layer-wise sensitivity is likewise not yet a completed ablation in the current manuscript. The aligned rerun supports first-divergence and autoregressive-propagation diagnostics, but its layer-trace outcome aggregates are too weak to serve as a standalone layer gate or proof of a low/mid/high-layer causal optimum. Any stronger late-layer amplification or layer-band optimum hypothesis should remain future work until measured directly.

Finally, the deployment-oriented evidence remains offline. The learned gates use online-observable features and avoid label leakage, but they still require held-out, time-split, and prospective online validation. Cost estimates are derived from current offline runs and feature logging; production latency, throughput, and memory overhead must be measured in the serving stack that actually enables AG.

9. Future Work: Validation, Statistics, and Mechanistic Tests

Several validation steps remain before stronger deployment claims are justified.

Model-family validation is the first requirement. Qwen3.5-9B and Qwen3-8B provide positive cross-architecture evidence, and Qwen2.5-7B provides a boundary case, but stronger claims require more family-internal validation. The next planned

tests are Qwen3.6 for the hybrid-attention family and additional full-attention Qwen-family comparisons.

Gate validation is the second requirement. Parameter overfitting has not been fully excluded. Learned gates must be validated on independent splits, time splits, and ideally new datasets. The current cross-domain ToolBench evidence is encouraging but not sufficient for broad claims.

Statistical validation is the third requirement. The tool feature-family and MC/choice reruns now include 95% bootstrap confidence intervals for their main comparisons. The remaining statistical step is broader coverage: repeat this validation for all category-level gates, additional held-out splits, repeated seeds, and future model routes rather than only the current rerun scopes.

Mechanistic validation is the fourth requirement. P1s/P2a/P2b require fuller tests. Current formal reports show that these refined points can be useful, but the hypotheses that P1s isolates gate-branch correction and that P2a/P2b are mechanistically orthogonal remain to be tested.

Online-gate validation is the fifth requirement. Continuous utility gates are promising but not yet final. Hard gates, mixed gates, ridge gates, and additive utility formulations should be compared under the same held-out, sample-level protocol before making deployment claims.

Training-time use of FFN signals is another open direction. This paper studies the inference-time form of FFN intervention: AG creates candidate forward paths and accepts them only when a gate predicts benefit. The same FFN-level signals may also be useful before deployment, for example to identify noisy tool-supervision examples, detect unstable schema or argument annotations, construct activation-aware curricula, or train gates with cleaner positive and negative examples. If FFN trajectories reveal where a model's structured decision becomes unstable, they may help improve both inference-time routing and the quality of the data used to train tool-structured behavior. We leave this training-side extension to future work.

10. Conclusion

AG emerged from a negative but informative ORP result. ORP showed that FFN intervention locations and e-values matter, but it also showed why direction-changing repair is unsafe: pretrained matrix directions should be treated as semantic structure, not as freely adjustable noise. AG keeps the diagnostic value of that result while removing the destructive operation. It keeps FFN weights fixed and modulates activation magnitudes at inference time.

The resulting method is not a universal quality booster. It is a targeted, gated inference-time intervention whose strongest current evidence lies in tool-structured generation. Qwen3.5-9B benefits most on function-call/tool-use tasks; Qwen3-8B benefits most on JSON and prefix-format structured output; and Qwen2.5-7B shows that oracle headroom without gate capture is not enough. The NS versus non-NS comparison reinforces the same point. Non-NS AG and Newton-Schulz-windowed NS AG behave as complementary candidate families rather than as a strict hierarchy: the former better captures sharp current-token amplitude decisions, while the latter can stabilize short-window FFN trajectories. Their relative value changes across the hybrid Qwen3.5-9B stack, the dense full-attention Qwen3-8B stack, and the wider-FFN Qwen2.5-7B boundary case. A deployable AG system should therefore expose both families and route between them rather than hard-coding either as the default.

Taken together, these results support a cautious but practical conclusion: non-destructive FFN amplitude gating is a promising route for safe LLM inference optimization, especially when deployed with model-specific, task-specific gates and strict fallback to baseline. The feature-family ablation strengthens that conclusion statistically for Qwen3/Qwen3.5 tool-route gates while also sharpening the negative boundary for Qwen2.5-7B and choice-style tasks.

The broader contribution is to establish FFN activations as a systematic intervention surface for structured LLM behavior: ORP exposes where the FFN is sensitive, AG shows how to intervene without changing model weights, and future training-time uses of FFN trajectories may extend the same signal from inference routing to data and supervision quality.

References

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention Is All You Need. NeurIPS. <https://arxiv.org/abs/1706.03762>
- [2] Shazeer, N. (2020). GLU Variants Improve Transformer. arXiv:2002.05202. <https://arxiv.org/abs/2002.05202>
- [3] Yang, A., Li, A., Yang, B., et al. (2025). Qwen3 Technical Report. arXiv:2505.09388. <https://arxiv.org/abs/2505.09388>
- [4] Yang, A., Yang, B., Zhang, B., et al. (2024). Qwen2.5 Technical Report. arXiv:2412.15115. <https://arxiv.org/abs/2412.15115>

- [5] DeepSeek-AI. (2026). DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. Project-archive technical note; retained as internal context, not as public or peer-reviewed evidence.
- [6] Qin, Y., Liang, S., Ye, Y., et al. (2023). ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. arXiv:2307.16789. <https://arxiv.org/abs/2307.16789>
- [7] Qu, C., Dai, S., Wei, X., Cai, H., Wang, S., Yin, D., Xu, J., and Wen, J.-R. (2024). Tool Learning with Large Language Models: A Survey. arXiv:2405.17935. <https://arxiv.org/abs/2405.17935>
- [8] Cobbe, K., Kosaraju, V., Bavarian, M., et al. (2021). Training Verifiers to Solve Math Word Problems. arXiv:2110.14168. <https://arxiv.org/abs/2110.14168>
- [9] Wang, Y., Ma, X., Zhang, G., et al. (2024). MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. arXiv:2406.01574. <https://arxiv.org/abs/2406.01574>
- [10] Huang, Y., Bai, Y., Zhu, Z., et al. (2023). C-Eval: A Multi-Level Multi-Discipline Chinese Evaluation Suite for Foundation Models. arXiv:2305.08322. <https://arxiv.org/abs/2305.08322>
- [11] Phan, L., Gatti, A., Han, Z., et al. (2025). Humanity's Last Exam. arXiv:2501.14249. <https://arxiv.org/abs/2501.14249>
- [12] Belcak, P., and Wattenhofer, R. (2023). Fast Feedforward Networks. arXiv:2308.14711. <https://arxiv.org/abs/2308.14711>
- [13] Zou, A., Phan, L., Wang, J., et al. (2023). Representation Engineering: A Top-Down Approach to AI Transparency. arXiv:2310.01405. <https://arxiv.org/abs/2310.01405>
- [14] Willard, B. T., and Louf, R. (2023). Efficient Guided Generation for Large Language Models. arXiv:2307.09702. <https://arxiv.org/abs/2307.09702>